Agatha Barbosa Marinho dos Santos
Bruna Kinjo Luiz Pinto
Kayke Ahrens Biscegli

Classificação Automática de Chamados de TI Utilizando Processamento de Linguagem Natural com Explicabilidade Baseada em Grafos

São Paulo
2026

Agatha Barbosa Marinho dos Santos
Bruna Kinjo Luiz Pinto
Kayke Ahrens Biscegli

Classificação Automática de Chamados de TI Utilizando Processamento de Linguagem Natural com Explicabilidade Baseada em Grafos

Trabalho de Conclusão de Curso II apresentado ao Senac Santo Amaro como requisito para conclusão do curso de Bacharelado em Ciência da Computação.

Senac – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Douglas José Baquião Ribeiro

São Paulo
2026

Resumo

O objetivo deste trabalho é investigar como a inclusão de uma camada de explicabilidade baseada em SHAP, LIME e grafos de co-ocorrência a um *pipeline* de classificação com BERTimbau contribui para a auditabilidade do processo decisório, comparando a qualidade das explicações dos dois métodos, utilizando chamados de suporte técnico em português brasileiro. A motivação é que classificadores baseados em *transformers* não revelam por que atribuíram uma categoria a um chamado, o que dificulta a auditoria de suas decisões. Os dados reúnem chamados anonimizados cedidos pela Bayer e pela Aché, num total de 53.279 chamados em 11 categorias, divididos em treino, validação e teste de modo que textos repetidos fiquem no mesmo conjunto. Foram comparados quatro cenários, que acrescentam um componente por vez: um *baseline* SVM com TF-IDF, o *fine-tuning* do BERTimbau, as explicações SHAP e LIME, e o grafo de co-ocorrência ponderado por essas explicações. Em uma amostra de 98 chamados do conjunto de teste, o SHAP e o LIME foram avaliados quanto à fidelidade, à estabilidade e à concordância entre si, e a rastreabilidade do grafo, tomada como critério de auditabilidade, foi examinada em estudos de caso. O BERTimbau obteve F1-macro de 0,628 e acurácia de 0,724 no conjunto de teste. O *baseline* SVM com TF-IDF obteve F1-macro de 0,591; a diferença do BERTimbau foi de 3,7 pontos percentuais (intervalo de confiança de 95%: 1,0 a 6,7), abaixo do critério de 5 pontos definido no TCC1. Nos 98 chamados, SHAP e LIME tiveram fidelidade bem acima do acaso e apontaram a mesma palavra no topo em 71% dos chamados; só o Tau-LOO distinguiu os métodos, a favor do LIME, e o SHAP estimado por amostragem foi menos estável que o LIME (mesma palavra no topo em 68% das repetições, contra 93%). Nos três estudos de caso, cada nó e cada aresta do grafo pôde ser levado ao chamado e aos pesos do SHAP e do LIME, e o grafo expôs, por exemplo, que um erro do modelo se apoiou em muito pouca evidência. Os resultados foram integrados a uma interface *web* que classifica um chamado digitado, exibe o grafo de co-ocorrência e permite alternar entre os pesos do SHAP e do LIME; a interface é uma prova de conceito e não foi avaliada junto a analistas de TI.

Palavras-chave: Processamento de Linguagem Natural. BERT. Inteligência Artificial Explicável. Grafos de Co-ocorrência. Classificação de Tickets de TI.

Abstract

The objective of this work is to investigate how adding an explainability layer based on SHAP, LIME and co-occurrence graphs to a BERTimbau classification *pipeline* contributes to the auditability of the decision process, comparing the quality of the explanations produced by the two methods, using technical support tickets in Brazilian Portuguese. The motivation is that *transformer*-based classifiers do not reveal why they assigned a category to a ticket, which makes it hard to audit their decisions. The data comprise anonymized tickets provided by Bayer and Aché, totaling 53,279 tickets in 11 categories, split into training, validation and test sets so that repeated texts remain in the same set. Four scenarios were compared, each adding one component: a *baseline* SVM with TF-IDF, the fine-tuning of BERTimbau, the SHAP and LIME explanations, and the co-occurrence graph weighted by those explanations. On a sample of 98 tickets from the test set, SHAP and LIME were evaluated for fidelity, stability and agreement with each other, and the traceability of the graph, taken as the criterion of auditability, was examined in case studies. BERTimbau achieved a macro F1 of 0.628 and an accuracy of 0.724 on the test set. The TF-IDF SVM *baseline* achieved a macro F1 of 0.591; the difference in favor of BERTimbau was 3.7 percentage points (95% confidence interval: 1.0 to 6.7), below the 5-point criterion set in the first phase of this work (TCC1). On the 98 tickets, SHAP and LIME showed fidelity well above chance and pointed to the same top word in 71% of tickets; only Tau-LOO separated the methods, favoring LIME, and sampled SHAP was less stable than LIME (same top word in 68% of repetitions versus 93%). In the three case studies, every node and edge of the graph could be traced back to the ticket and to the SHAP and LIME weights, and the graph showed, for example, that a model error relied on very little evidence. The results were integrated into a web interface that classifies a typed ticket, displays the co-occurrence graph and allows switching between SHAP and LIME weights; the interface is a proof of concept and was not evaluated with IT analysts.

Keywords: Natural Language Processing. BERT. Explainable Artificial Intelligence. Co-occurrence Graphs. IT Ticket Classification.

Lista de ilustrações

Figura 1 – Arquitetura *Transformer* simplificada 25
Figura 2 – Exemplo ilustrativo de *self-attention* para o *token* “banco” 26
Figura 3 – Atenção unidirecional e atenção bidirecional 27
Figura 4 – Exemplo de explicação gerada pelo LIME 32
Figura 5 – Representação dos valores SHAP por *feature* 34
Figura 6 – Protótipo de baixa fidelidade da interface (proposta 1) 51
Figura 7 – Protótipo de baixa fidelidade da interface (proposta 2) 51
Figura 8 – Protótipo de baixa fidelidade da interface (proposta 3) 52
Figura 9 – Protótipo de baixa fidelidade da interface final - Tela do grafo 52
Figura 10 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas . 53
Figura 11 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência . 54
Figura 12 – Protótipo de média fidelidade - visualização estatística dos *tokens* . . . 54
Figura 13 – Interface final: classificação de um chamado digitado e grafo de co-ocorrência 55
Figura 14 – Matriz de confusão do BERTimbau no conjunto de teste, normalizada por classe real 63
Figura 15 – Grafo de co-ocorrência do caso 1 (ache_ticket_001118), com pesos SHAP 67
Figura 16 – Grafo de co-ocorrência do caso 2 (bayer_ticket_001351), com pesos SHAP 68
Figura 17 – Grafo de co-ocorrência do caso 3 (bayer_ticket_016056), com pesos SHAP 68
Figura 18 – Autorização de uso de dados concedida pela Aché Laboratórios Farmacêuticos (1 de 2): pedido e concordância 80
Figura 19 – Autorização de uso de dados concedida pela Aché Laboratórios Farmacêuticos (2 de 2): acompanhamento e envio da base 81
Figura 20 – Autorização de uso de dados concedida pela Bayer S.A. 82

Lista de tabelas

Tabela 1 – Exemplo ilustrativo de *embeddings* de quatro dimensões 24
Tabela 2 – Comparação entre trabalhos relacionados 41
Tabela 3 – Cenários experimentais para avaliação do sistema 46
Tabela 4 – Bibliotecas utilizadas em cada etapa do projeto 47
Tabela 5 – Categorias de classificação dos chamados 60
Tabela 6 – Desempenho dos classificadores no conjunto de teste 61
Tabela 7 – F1 por classe no conjunto de teste e diferença entre BERTimbau e SVM 62
Tabela 8 – Desempenho no conjunto de teste, por empresa de origem 62
Tabela 9 – Fidelidade das explicações do SHAP e do LIME nos 98 chamados da amostra 64
Tabela 10 – Estabilidade das explicações do SHAP e do LIME em 12 chamados, com 5 repetições 65
Tabela 11 – Concordância entre SHAP e LIME nos 98 chamados da amostra 65
Tabela 12 – Concordância entre SHAP e LIME por subconjunto de chamados . . . 66
Tabela 13 – Chamados usados nos estudos de caso do grafo 67
Tabela 14 – Cronograma do TCC2, com datas e status 78

Lista de abreviaturas e siglas

AOPC	*Area Over the Perturbation Curve* (Área sobre a Curva de Perturbação)
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BEGNN	*BERT-Enhanced Graph Neural Network* (Rede Neural de Grafos Aprimorada por BERT)
BERT	*Bidirectional Encoder Representations from Transformers* (Representações de Codificador Bidirecional a partir de Transformers)
BERTimbau	Variante do BERT pré-treinada para o português brasileiro
BiLSTM	*Bidirectional Long Short-Term Memory* (Memória de Longo e Curto Prazo Bidirecional)
CNN	*Convolutional Neural Network* (Rede Neural Convolucional)
CPU	*Central Processing Unit* (Unidade Central de Processamento)
CSS	*Cascading Style Sheets* (Folhas de Estilo em Cascata)
GCN	*Graph Convolutional Network* (Rede Neural Convolucional em Grafos)
GloVe	*Global Vectors for Word Representation* (Vetores Globais para Representação de Palavras)
GNN	*Graph Neural Network* (Rede Neural de Grafos)
GPT	*Generative Pre-trained Transformer* (Transformer Generativo Pré-treinado)
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
HOOM	*Hybrid Online Offline Model* (Modelo Híbrido *Online* e *Offline*)
ITIL	*Information Technology Infrastructure Library* (Biblioteca de Infraestrutura de Tecnologia da Informação)
ITSM	*IT Service Management* (Gerenciamento de Serviços de TI)
JSON	*JavaScript Object Notation* (Notação de Objetos JavaScript)
LGPD	Lei Geral de Proteção de Dados Pessoais

LIME	*Local Interpretable Model-Agnostic Explanations* (Explicações Locais Interpretáveis Independentes de Modelo)
LOO	*Leave-One-Out* (Deixar Um de Fora)
LSTM	*Long Short-Term Memory* (Memória de Longo e Curto Prazo)
mBERT	*Multilingual BERT* (BERT Multilíngue)
MLM	*Masked Language Modeling* (Modelagem de Linguagem com Máscara)
MLP	*Multilayer Perceptron* (Perceptron Multicamadas)
NER	*Named Entity Recognition* (Reconhecimento de Entidades Nomeadas)
NLTK	*Natural Language Toolkit* (Kit de Ferramentas de Linguagem Natural)
NSP	*Next Sentence Prediction* (Predição da Próxima Sentença)
PLN	Processamento de Linguagem Natural
PMI	*Pointwise Mutual Information* (Informação Mútua Pontual)
PT-BR	Português Brasileiro
RNN	*Recurrent Neural Network* (Rede Neural Recorrente)
RoBERTa	*Robustly Optimized BERT Pretraining Approach* (Abordagem de Pré-treinamento do BERT Robustamente Otimizada)
SHAP	*SHapley Additive exPlanations* (Explicações Aditivas de Shapley)
SLA	*Service Level Agreement* (Acordo de Nível de Serviço)
SMOTE	*Synthetic Minority Oversampling Technique* (Técnica de Sobreamostragem Sintética de Minorias)
SVM	*Support Vector Machine* (Máquina de Vetores de Suporte)
TF-IDF	*Term Frequency–Inverse Document Frequency* (Frequência do Termo–Inverso da Frequência nos Documentos)
TI	Tecnologia da Informação
XAI	*Explainable Artificial Intelligence* (Inteligência Artificial Explicável)
XLM-RoBERTa	*Cross-lingual Language Model RoBERTa* (Modelo de Linguagem Multilíngue RoBERTa)
ZHAW	*Zürcher Hochschule für Angewandte Wissenschaften* (Universidade de Ciências Aplicadas de Zurique)

Sumário

1 INTRODUÇÃO 12
1.1 Contextualização do problema 12
1.2 Formulação do problema 14
1.3 Objetivos e hipótese 14
1.3.1 Objetivo geral 14
1.3.2 Objetivos específicos 14
1.3.3 Hipótese de pesquisa 15
1.4 Justificativa 15

2 REVISÃO BIBLIOGRÁFICA 17
2.1 Metodologia da pesquisa bibliográfica 17
2.2 Fundamentos teóricos 17
2.2.1 Sistema de *Help Desk* e Triagem de Chamados 17
2.2.2 Processamento de Linguagem Natural 17
2.2.3 Modelos de Linguagem Baseados em *Transformer* 18
2.2.4 Inteligência Artificial Explicável 18
2.2.5 Grafo de Co-ocorrência 18
2.2.6 Auditabilidade 18
2.3 Técnicas, métodos e abordagens existentes 19
2.3.1 Definição e importância dos sistemas de *help desk* nas organizações 19
2.3.2 O processo manual de triagem e seus problemas 20
2.3.3 Abordagens de aprendizado de máquina para classificação automática de *tickets* 21
2.3.3.1 Modelos clássicos — *Naive Bayes* e SVM com TF-IDF 21
2.3.3.2 Redes neurais convolucionais aplicadas a *tickets* 22
2.3.3.3 Classificação de chamados no contexto brasileiro 22
2.3.4 Representações vetoriais de texto 23
2.3.4.1 *TF-IDF* e modelos baseados em frequência 23
2.3.4.2 *Embeddings* estáticos — *Word2Vec* e *GloVe* 23
2.3.4.3 Limitações das representações estáticas — polissemia e contexto 24
2.3.5 Arquitetura *Transformer* e mecanismo de atenção 24
2.3.5.1 *Self-attention* e atenção bidirecional 24
2.3.5.2 Pré-treinamento — *Masked Language Modeling* e *Next Sentence Prediction* . . 27
2.3.5.3 *Fine-tuning* para tarefas *downstream* 27
2.3.6 BERT 28

2.3.6.1 Arquitetura e configurações *BERT-base* e *BERT-large* . . . 28
2.3.6.2 *Token* [CLS] e classificação de sequências . . . 28
2.3.7 BERTimbau — BERT para o português brasileiro . . . 28
2.3.7.1 Corpus de pré-treinamento e diferenças em relação ao mBERT . . . 28
2.3.7.2 Desempenho em tarefas de PLN em PT-BR . . . 29
2.3.8 *Fine-tuning* de BERT para classificação de chamados de TI . . . 29
2.3.9 Definição e motivação . . . 30
2.3.9.1 O problema da caixa-preta em aprendizado profundo . . . 30
2.3.9.2 Confiança, explicabilidade e adoção em produção . . . 30
2.3.9.3 Taxonomia de métodos XAI — intrínsecos versus *post-hoc*, locais versus globais 31
2.3.10 LIME — *Local Interpretable Model-Agnostic Explanations* . . . 31
2.3.10.1 Funcionamento — perturbações e modelo local interpretável . . . 31
2.3.10.2 Aplicação em classificação de texto . . . 32
2.3.10.3 Vantagens e limitações . . . 33
2.3.11 SHAP — *SHapley Additive exPlanations* . . . 33
2.3.11.1 Fundamentos — valores de Shapley da teoria dos jogos . . . 33
2.3.11.2 Propriedades matemáticas — eficiência, simetria e ausência . . . 34
2.3.11.3 Cálculo do SHAP em modelos de texto . . . 34
2.3.12 Comparação entre SHAP e LIME . . . 35
2.3.12.1 Consistência e estabilidade das explicações . . . 35
2.3.12.2 Custo computacional . . . 35
2.3.12.3 Justificativa para uso conjunto neste trabalho . . . 35
2.3.13 XAI aplicado a classificação de texto e tickets de TI . . . 35
2.3.13.1 LIME em classificação multiclasse de texto . . . 35
2.3.13.2 SHAP e LIME em ambiente ITSM corporativo . . . 36
2.3.14 Fundamentos de grafos aplicados a texto . . . 36
2.3.14.1 Definição de grafo de co-ocorrência — nós, arestas e janela deslizante . . . 36
2.3.14.2 Ponderação de arestas — frequência, PMI . . . 37
2.3.15 *TextGCN* — formalização do grafo de co-ocorrência . . . 38
2.3.15.1 Grafo heterogêneo documento-palavra . . . 38
2.3.15.2 Redes neurais de grafos e propagação de informação . . . 38
2.3.16 Combinação de BERT com grafos de co-ocorrência . . . 38
2.3.16.1 *Embeddings* BERT como *features* iniciais dos nós . . . 38
2.3.16.2 Resultados em vocabulário técnico especializado . . . 39
2.3.17 Grafo de co-ocorrência combinado com XAI . . . 39
2.3.17.1 Ponderação de nós por *scores* de explicabilidade . . . 39
2.3.17.2 Visualização interativa como camada de auditabilidade . . . 39
2.4 Trabalhos correlatos e limitações . . . 40
2.4.1 Trabalho 1: Text Classification of Service Desk Tickets . . . 40

2.4.2 Trabalho 2: A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets) . . . 40
2.4.3 Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chamados de suporte . . . 41
2.4.4 Trabalho 4: AI-Based Classification of IT Support Requests in Enterprise Service Management Systems . . . 41
2.4.5 Limitações dos modelos existentes . . . 42
2.4.5.1 O debate BERT versus SVM em domínios especializados . . . 42
2.4.5.2 A caixa-preta e a verificação das decisões em produção . . . 42
2.4.6 Lacunas identificadas no estado da arte . . . 43
2.4.6.1 Ausência de XAI em sistemas de classificação de tickets em produção . . . 43
2.4.6.2 Carência de soluções robustas para PT-BR em *helpdesk* . . . 43
2.4.6.3 Custo-benefício do BERT sem camada de explicabilidade . . . 43
2.4.7 Direcionamento para a proposta . . . 44

3 METODOLOGIA . . . 45
3.1 Tipo de pesquisa e hipótese . . . 45
3.2 Procedimentos da abordagem proposta . . . 45
3.2.1 Cenários experimentais . . . 46
3.3 Ferramentas, tecnologias e ambientes . . . 46
3.4 Procedimentos experimentais e métricas . . . 47
3.4.1 Métricas de desempenho do classificador . . . 47
3.4.2 Critérios de validação . . . 48

4 DESENVOLVIMENTO . . . 50
4.1 Modelagem e arquitetura . . . 50
4.1.1 Visão geral da solução . . . 50
4.1.2 Protótipos da interface . . . 50
4.1.3 Arquitetura da solução . . . 55
4.2 Detalhes de implementação . . . 56
4.2.1 Tecnologias e ferramentas . . . 56
4.2.2 Funcionalidades principais . . . 57
4.2.3 Metodologia de implementação . . . 57
4.3 Experimentos e cenários de teste . . . 58
4.3.1 Cenários A e B: classificação . . . 58
4.3.2 Cenário C: explicações SHAP e LIME . . . 58
4.3.3 Cenário D: grafo de co-ocorrência . . . 58
4.4 Coleta e tratamento de dados . . . 59
4.4.1 Origem e autorização de uso dos dados . . . 59

4.4.2 *Dataset* e *fine-tuning* 59

5 RESULTADOS E ANÁLISE 61
5.1 Apresentação dos resultados 61
5.1.1 Desempenho da classificação (cenários A e B) 61
5.1.2 Explicabilidade: SHAP e LIME (cenário C) 63
5.1.2.1 Fidelidade 63
5.1.2.2 Estabilidade 64
5.1.2.3 Concordância entre SHAP e LIME 65
5.1.3 Grafo de co-ocorrência (cenário D) 66
5.2 Análise crítica e comparação 69
5.3 Discussão 70

6 CONCLUSÃO 71
6.1 Retomada do problema e dos objetivos 71
6.2 Contribuições para a Computação 72
6.3 Limitações e trabalhos futuros 72

REFERÊNCIAS 74

APÊNDICES 77

APÊNDICE A – CRONOGRAMA DE ATIVIDADES 78

ANEXOS 79

ANEXO A – AUTORIZAÇÃO DE USO DE DADOS — ACHÉ LABORATÓRIOS FARMACÊUTICOS 80

ANEXO B – AUTORIZAÇÃO DE USO DE DADOS — BAYER S.A. 82

1 Introdução

Os *helpdesks* das organizações recebem grandes volumes de chamados de suporte de Tecnologia da Informação (TI): empresas modernas geram milhares de chamados por mês (RAZMA; JURKUS, 2026), e cada um precisa ser lido, classificado em uma categoria e encaminhado à equipe responsável. Os dados deste trabalho dão a dimensão: a Aché registrou 9.317 chamados em um único mês (abril de 2026), e a Bayer, 191.350 entre janeiro de 2021 e agosto de 2026. Quando essa triagem é manual, ela é lenta e inconsistente, e os chamados trazem descrições variadas, com jargão técnico, erros de ortografia e expressões coloquiais, o que a torna ainda mais difícil (ZEMP, 2021).

O custo de uma triagem ruim recai sobre toda a organização. Um chamado classificado na categoria errada vai para uma equipe sem competência para resolvê-lo, e o ciclo de reatribuições aumenta o tempo de resolução, desperdiça o tempo da equipe e frustra o usuário (HARUN; HUSPI; IAHAD, 2023). Esse atraso pode comprometer o cumprimento do SLA (*Service Level Agreement* – Acordo de Nível de Serviço), o prazo de resolução acordado com o usuário, com impacto na produtividade da organização e na satisfação dos usuários (RAZMA; JURKUS, 2026). A triagem manual também depende do conhecimento de cada analista sobre as categorias da empresa, conhecimento que se perde com a rotatividade (WAHBA, 2023), e, no Brasil, o problema é agravado pela informalidade da linguagem dos usuários (PEREIRA et al., 2023).

Por isso, a classificação automática de chamados é estudada com Processamento de Linguagem Natural (PLN) e já chegou à prática: no estudo de Zemp (2021), com mais de um milhão de chamados de um banco, uma CNN (*Convolutional Neural Network* – Rede Neural Convolucional) alcançou 92% de acurácia de validação, e a previsão do serviço afetado foi considerada útil, por unanimidade, pela equipe de suporte. Os modelos mais recentes, como o BERT (*Bidirectional Encoder Representations from Transformers* – Representações de Codificador Bidirecional a partir de Transformers), têm mais de cem milhões de parâmetros (110 milhões no BERT-base (DEVLIN et al., 2019)) e funcionam como caixas-pretas: produzem a categoria sem revelar quais palavras do chamado a determinaram. Sem essa informação, o analista não consegue conferir a decisão, nem o gestor identificar erros sistemáticos, e o bom desempenho sozinho não garante a adoção: no mesmo estudo, a equipe de suporte recusou a função que separava requisições de incidentes, mesmo com 93% de acurácia de validação (Seção 2.4.5.2).

1.1 Contextualização do problema

Neste contexto, explicabilidade é a capacidade de mostrar quais palavras do chamado levaram o modelo à categoria escolhida. A área de XAI (*Explainable Artificial Intelligence* –

Inteligência Artificial Explicável) reúne métodos para isso, como o SHAP (*SHapley Additive exPlanations* – Explicações Aditivas de Shapley), que distribui o crédito da decisão entre as palavras com base na teoria dos jogos, e o LIME (*Local Interpretable Model-Agnostic Explanations* – Explicações Locais Interpretáveis Independentes de Modelo), que altera o texto e ajusta um modelo simples para ver quais palavras mais pesam (LUNDBERG; LEE, 2017; RIBEIRO; SINGH; GUESTRIN, 2016). Os dois atribuem uma importância a cada palavra do chamado.

Essa informação serve a diferentes interessados. O analista pode verificar se os termos destacados fazem sentido para a categoria e decidir se confia na previsão (RIBEIRO; SINGH; GUESTRIN, 2016; RAZMA; JURKUS, 2026). O gestor pode auditar o sistema e identificar erros sistemáticos, por exemplo uma categoria que o modelo decide por palavras irrelevantes. A equipe que mantém o modelo pode depurá-lo e melhorá-lo: no estudo de Mustafa e Hama Saeed (2025), as explicações do LIME revelaram que palavras sem conteúdo informativo recebiam pesos altos, o que levou a ajustar o pré-processamento. E o usuário, indiretamente, se beneficia de chamados mais bem encaminhados.

Mas uma explicação só ajuda se for confiável e legível, e há dois obstáculos. O primeiro é que o SHAP e o LIME podem divergir entre si e variar entre execuções, pois o LIME depende de amostragem aleatória (SALIH et al., 2023), o que torna necessário medir a fidelidade e a estabilidade das explicações. O segundo é que uma lista de pesos por palavra mostra a importância de cada termo, mas não como eles se relacionam no texto. Um grafo de co-ocorrência, em que cada nó é uma palavra e cada aresta liga palavras que aparecem próximas (YAO; MAO; LUO, 2019), pode apresentar as explicações de forma visual e rastreável (TALEBI; ABDOLVAND, 2025).

Em português brasileiro, esse tema é pouco explorado. Os trabalhos encontrados sobre classificação de chamados em português reportam 75% de precisão média, em sete categorias (PEREIRA et al., 2023), e F1-macro de até 61%, em 226.438 chamados (YOKOTA, 2025), e nenhum aplica modelos BERT com explicações. Fora do português, Razma e Jurkus (2026) usaram o LIME e o SHAP sobre chamados de TI, mas mostraram as explicações em exemplos, sem medir sua fidelidade ou estabilidade e sem estruturá-las em grafo. Não foi identificado trabalho que reúna a classificação com BERT, a comparação entre SHAP e LIME e o grafo de co-ocorrência para chamados de TI em português (Seção 2.4).

É essa lacuna que este trabalho busca preencher. A proposta é classificar chamados com o BERTimbau, uma versão do BERT para o português brasileiro (SOUZA; NOGUEIRA; LOTUFO, 2020), explicar cada decisão com SHAP e LIME, comparar os dois métodos quanto à fidelidade e à estabilidade e apresentar as explicações em um grafo de co-ocorrência, para avaliar se ele permite rastrear cada decisão. Para isso, foram usados 53.279 chamados reais, em 11 categorias, cedidos pela Aché e pela Bayer. Os objetivos e a hipótese estão na Seção 1.3.3.

1.2 Formulação do problema

Classificadores baseados em *transformers* alcançam bom desempenho na triagem de chamados de TI, mas não revelam por que atribuíram uma categoria a um chamado. Existem métodos que explicam essas decisões, como SHAP e LIME, mas ainda é preciso verificar se, para chamados em português brasileiro, essas explicações, apresentadas em forma de grafo, permitem a um analista acompanhar o caminho até a decisão.

Diante disso, este trabalho busca responder à seguinte pergunta: **em chamados de suporte de TI em português brasileiro, como se comparam o SHAP e o LIME quanto à fidelidade e à estabilidade das explicações de um classificador BERTimbau, e de que forma um grafo de co-ocorrência contribui para tornar o processo de classificação auditável?**

1.3 Objetivos e hipótese

1.3.1 Objetivo geral

Investigar como uma camada de explicabilidade (SHAP e LIME) e grafos de co-ocorrência, acrescentada a um *pipeline* de classificação baseado em *transformers*, contribui para a auditabilidade do processo decisório, comparando a qualidade das explicações dos dois métodos, utilizando chamados de suporte de TI em português brasileiro.

1.3.2 Objetivos específicos

1. Analisar o ganho de desempenho do modelo BERTimbau com *fine-tuning* em relação ao *baseline* SVM com TF-IDF, utilizando a métrica F1-macro.

2. Comparar a explicabilidade gerada por SHAP e LIME em termos de fidelidade, estabilidade, concordância entre os métodos e relevância semântica, calculando essas medidas sobre uma amostra de chamados do conjunto de teste: a fidelidade por meio da *comprehensiveness*, da *sufficiency* e do Tau-LOO; a estabilidade, repetindo o cálculo com sorteios diferentes; e a concordância, comparando as palavras mais relevantes apontadas pelos dois métodos.

3. Analisar se a visualização em grafos de co-ocorrência ponderados por valores SHAP e LIME fornece auditabilidade ao processo de classificação, no sentido de rastrear cada decisão até os termos, a janela de palavras e os valores SHAP e LIME que a originaram. Para isso, constrói-se um grafo para cada chamado, com as palavras como nós, ponderados pelos valores SHAP e LIME, e a co-ocorrência na janela como arestas, e verifica-se, em estudos de caso, se cada nó e cada aresta pode ser levado ao chamado e aos pesos das explicações.

4. Validar a consistência semântica das explicações geradas pelo *pipeline* no contexto de chamados de suporte técnico em português brasileiro, verificando se os termos destacados correspondem ao vocabulário técnico esperado para cada categoria classificada.

1.3.3 Hipótese de pesquisa

A utilização de modelagem em grafos, ponderados pelos valores SHAP e LIME, como camada de explicabilidade visual para o modelo BERT contribui para a auditabilidade do processo de classificação automatizada, permitindo rastrear quais termos do chamado contribuíram para a categoria atribuída pelo modelo. Os métodos SHAP e LIME, por sua vez, produzem explicações com fidelidade e estabilidade distintas.

1.4 Justificativa

A triagem de chamados é uma etapa de grande volume nas empresas. Feita manualmente, é lenta, depende da experiência de cada analista e está sujeita a inconsistências, e o atraso pode comprometer o cumprimento do SLA, como discutido na Introdução. A classificação automática com PLN já é estudada e usada na prática, o que mostra que o problema é relevante e que há ganho possível.

Esse problema, no entanto, está mal resolvido em dois pontos. Os modelos que melhor classificam, baseados em *transformers*, funcionam como caixas-pretas, e um analista ou auditor não consegue verificar por que um chamado foi para uma categoria. E, para o português brasileiro, os trabalhos encontrados não aplicam modelos BERT com explicações, e não foi identificado trabalho que reúna a classificação com BERT, a comparação entre SHAP e LIME e o grafo de co-ocorrência para chamados de suporte.

Este trabalho propõe acrescentar ao classificador uma camada de explicabilidade: SHAP e LIME indicam as palavras que levaram à categoria, e um grafo de co-ocorrência ponderado por esses valores mostra como essas palavras se relacionam no chamado. Para o analista, isso permite conferir a decisão do modelo. Para o gestor, permite auditar a triagem e identificar erros do modelo. Para quem desenvolve o modelo, permite encontrar palavras sem valor informativo que pesam na decisão, como já observado por Mustafa e Hama Saeed (2025).

A contribuição é prática e científica. Do lado prático, há uma prova de conceito de interface para chamados em português. Do lado científico, compara-se SHAP e LIME em um idioma e um domínio pouco estudados e mede-se o ganho do BERTimbau sobre o *baseline* SVM com TF-IDF, relevante porque Wahba (2023) questiona o custo-benefício do BERT.

O restante deste documento está organizado da seguinte forma. O Capítulo 2 apresenta a revisão bibliográfica: os fundamentos teóricos, as técnicas existentes e os trabalhos

correlatos. O Capítulo 3 descreve a metodologia e as métricas de avaliação. O Capítulo 4 detalha a solução, a implementação, os experimentos e os dados. O Capítulo 5 apresenta e analisa os resultados, e o Capítulo 6 traz as conclusões, as limitações e os trabalhos futuros.

2 Revisão Bibliográfica

Este capítulo apresenta a base teórica e o estado da arte do trabalho. Ele começa pela metodologia da pesquisa bibliográfica (Seção 2.1) e pelos fundamentos teóricos necessários à leitura (Seção 2.2): Processamento de Linguagem Natural, modelos baseados em *Transformer*, Inteligência Artificial Explicável, grafos de co-ocorrência e auditabilidade. Em seguida, aprofunda as técnicas, métodos e abordagens existentes (Seção 2.3) e encerra com os trabalhos correlatos, as limitações da literatura e o posicionamento deste trabalho (Seção 2.4).

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi realizada nas bases Google Scholar, IEEE Xplore, arXiv e SciELO, utilizando as palavras-chave “BERT”, “Grafos”, “XAI”, “PLN”, “Help Desk”, “Chamados de TI”, “SHAP”, “LIME”, “Tickets”, “Support”, “Pipeline”, “Classificação”, “Caixa Preta” e “Knowledge Graph”, entre outras. Foram selecionados artigos publicados entre 2021 e 2026, em português e inglês, que tivessem relação direta com o tema abordado.

2.2 Fundamentos teóricos

Esta seção apresenta os conceitos técnicos fundamentais que sustentam a proposta deste trabalho. O objetivo é fornecer ao leitor as definições necessárias para compreender a solução proposta antes de sua descrição detalhada.

2.2.1 Sistema de *Help Desk* e Triagem de Chamados

Um sistema de *help desk* é uma plataforma que centraliza e gerencia solicitações de suporte técnico submetidas pelos usuários de uma organização. A triagem de chamados consiste em classificar cada solicitação em uma categoria predefinida e encaminhá-la à equipe responsável pela resolução. A automação desse processo, substituindo ou auxiliando a triagem manual, é o problema central abordado por este trabalho (ZEMP, 2021).

2.2.2 Processamento de Linguagem Natural

Processamento de Linguagem Natural (PLN) é o campo da inteligência artificial dedicado ao desenvolvimento de métodos computacionais para compreender, interpretar e gerar linguagem humana. No contexto deste trabalho, PLN é utilizado para representar chamados de suporte técnico como vetores numéricos que um modelo de aprendizado de

máquina possa processar, e para extrair informações semânticas dos textos submetidos pelos usuários (DEVLIN et al., 2019).

2.2.3 Modelos de Linguagem Baseados em *Transformer*

Modelos de linguagem baseados na arquitetura *Transformer* (VASWANI et al., 2017) aprendem representações contextualizadas de texto por meio de mecanismos de atenção que consideram simultaneamente todas as posições de uma sequência de entrada. O BERT (DEVLIN et al., 2019) é o modelo desta família mais amplamente adotado para tarefas de classificação de texto. O BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020) é uma variante do BERT pré-treinada especificamente sobre corpora em português brasileiro, sendo o modelo adotado neste trabalho.

2.2.4 Inteligência Artificial Explicável

Inteligência Artificial Explicável (XAI) refere-se ao conjunto de métodos e técnicas que permitem compreender e auditar as decisões de modelos de aprendizado de máquina (SALIH et al., 2023). Dois métodos *post-hoc* foram utilizados neste trabalho:

- **LIME** (RIBEIRO; SINGH; GUESTRIN, 2016): explica predições individuais ajustando um modelo linear simples em torno de perturbações da entrada original, indicando quais palavras mais contribuíram para a classificação.
- **SHAP** (LUNDBERG; LEE, 2017): atribui a cada palavra da entrada um valor de importância calculado com base na teoria dos jogos cooperativos, com garantias matemáticas de consistência e equidade na distribuição de crédito.

2.2.5 Grafo de Co-ocorrência

Um grafo de co-ocorrência é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$ na qual os nós V representam os termos únicos de um texto e as arestas E conectam pares de termos que aparecem juntos dentro de uma janela deslizante de tamanho fixo (YAO; MAO; LUO, 2019). Neste trabalho, essa estrutura foi utilizada como camada de visualização das explicações XAI: os pesos dos nós foram determinados pelos valores SHAP e LIME de cada *token*, e os pesos das arestas, pela frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo.

2.2.6 Auditabilidade

A literatura de XAI não apresenta uma definição única e consolidada de auditabilidade. Por essa razão, adota-se aqui uma definição operacional, construída a partir de propriedades

já estabelecidas na literatura: um sistema de classificação é auditável quando permite rastrear, verificar e reproduzir a explicação de cada decisão do modelo. Trata-se, portanto, de uma propriedade técnica do sistema, e não da confiança percebida por quem o utiliza. Essa distinção é relevante porque o estudo de Zemp (2021), ao relatar a reação de analistas de suporte a um sistema de classificação em produção, trata da confiança percebida, e não da propriedade técnica definida aqui.

A definição se desdobra em três dimensões:

- **Rastreabilidade**: a capacidade de atribuir a decisão do modelo a termos específicos da entrada, obtida por meio dos valores SHAP (LUNDBERG; LEE, 2017) e LIME (RIBEIRO; SINGH; GUESTRIN, 2016);
- **Estabilidade**: a reprodutibilidade da explicação gerada para uma mesma instância ao longo de execuções repetidas, questão especialmente relevante para o LIME, que depende de amostragem aleatória de perturbações (SALIH et al., 2023);
- **Fidelidade**: o grau em que a explicação reflete o processo de decisão efetivamente utilizado pelo modelo (JACOVI; GOLDBERG, 2020), verificável pela variação da confiança preditiva ao remover da entrada os termos apontados como mais relevantes.

A validação da percepção de confiança por parte de gestores ou analistas de TI não é objeto deste trabalho, sendo reconhecida como extensão natural da pesquisa.

2.3 Técnicas, métodos e abordagens existentes

As subseções a seguir aprofundam os conceitos apresentados na Seção 2.2 e descrevem as abordagens já empregadas na classificação de chamados de TI, nos modelos de linguagem, na explicabilidade e nos grafos de co-ocorrência.

2.3.1 Definição e importância dos sistemas de *help desk* nas organizações

Sistemas de *help desk*, também denominados sistemas de suporte técnico, centrais de serviço ou plataformas ITSM (*IT Service Management* – Gerenciamento de Serviços de TI), são estruturas organizacionais e tecnológicas responsáveis por centralizar e gerenciar as solicitações de suporte submetidas pelos usuários de uma organização. Esses sistemas recebem chamados relacionados a falhas de infraestrutura, problemas em aplicações, solicitações de acesso, dúvidas operacionais e demais ocorrências que interrompem ou dificultam o trabalho dos colaboradores (ZEMP, 2021).

A importância desses sistemas para a continuidade operacional das organizações é amplamente reconhecida na literatura. Razma e Jurkus (2026) destacam que empresas modernas geram milhares de chamados de suporte mensalmente, e que cada chamado deve

ser corretamente categorizado e encaminhado à equipe responsável para que o prazo de resolução estabelecido no SLA (*Service Level Agreement* – Acordo de Nível de Serviço) seja cumprido. O não cumprimento desses prazos impacta diretamente a produtividade organizacional e a satisfação dos usuários.

Na prática, um chamado percorre um ciclo. O usuário o abre por um portal, *e-mail*, telefone ou *chat*, descrevendo o problema; o sistema registra a descrição, a data e o solicitante; o chamado é então classificado em uma categoria e, conforme ela, priorizado e encaminhado à equipe responsável (ZEMP, 2021); por fim, a equipe resolve e encerra o chamado, enquanto o sistema acompanha o prazo do SLA durante todo o processo. Nos dados deste trabalho, cada chamado traz a descrição completa e uma categoria, e os da Bayer trazem também uma descrição curta (Seção 4.4.2). A etapa de classificação é a que este trabalho busca automatizar e explicar.

Boa parte dessa organização vem do ITIL (*Information Technology Infrastructure Library* – Biblioteca de Infraestrutura de Tecnologia da Informação), um conjunto de boas práticas de gerenciamento de serviços de TI amplamente adotado (AXELOS, 2019). O ITIL distingue o incidente, uma interrupção não planejada de um serviço ou redução de sua qualidade, da requisição de serviço, um pedido do usuário que inicia uma ação de serviço acordada como parte normal da entrega do serviço (AXELOS, 2019). Essa é a distinção que a equipe de suporte do estudo de Zemp (2021) recusou automatizar (Seção 2.4.5.2).

Os sistemas que implementam esse ciclo podem ser de código aberto ou comerciais. Entre os de código aberto estão o GLPI, o Znuny, o Zammad e o osTicket, que a própria organização pode instalar e adaptar; entre os comerciais estão o ServiceNow, o Jira Service Management, o Zendesk e o Freshservice, oferecidos como serviço pago, em geral na nuvem. Independentemente do produto, esses sistemas em geral guardam o texto de cada chamado e sua categoria, e é esse histórico que permite treinar classificadores como os deste trabalho.

2.3.2 O processo manual de triagem e seus problemas

Em grande parte das organizações, a triagem de chamados ainda é realizada manualmente por analistas de suporte, que leem a descrição textual do problema submetida pelo usuário, determinam a categoria correta e encaminham o chamado à equipe técnica responsável (ZEMP, 2021). Esse processo apresenta limitações estruturais que se tornam mais evidentes à medida que o volume de chamados cresce.

A inconsistência na categorização é um dos principais problemas. Analistas diferentes podem categorizar o mesmo tipo de chamado de formas distintas, e um mesmo analista pode ser inconsistente ao longo do tempo. Além disso, a triagem manual depende do conhecimento individual de cada analista sobre a estrutura de categorias da organização, conhecimento que não é facilmente transferível e que se perde quando há rotatividade de pessoal (WAHBA, 2023).

Chamados categorizados incorretamente são encaminhados para equipes que não possuem competência para resolvê-los, isso gera um ciclo de reatribuição de chamados que eleva o tempo de resolução, desperdiça o tempo da equipe e deixa o usuário frustrado (HARUN; HUSPI; IAHAD, 2023). Pereira et al. (2023) observam que, no contexto brasileiro, esse problema é agravado pela informalidade da linguagem utilizada pelos usuários ao descrever seus problemas, o que dificulta ainda mais a categorização manual.

2.3.3 Abordagens de aprendizado de máquina para classificação automática de *tickets*

A automação da triagem de chamados por meio de aprendizado de máquina tem sido investigada de forma crescente na literatura. O problema é tratado como uma tarefa de classificação supervisionada de texto: dado o histórico de chamados já categorizados, treina-se um modelo capaz de predizer a categoria de novos chamados a partir de sua descrição textual.

2.3.3.1 Modelos clássicos — *Naive Bayes* e SVM com TF-IDF

As primeiras abordagens automatizadas para classificação de chamados utilizaram algoritmos clássicos de aprendizado de máquina combinados com representações baseadas em frequência de termos. Harun, Huspi e Iahad (2023) investigaram a classificação automática de perguntas em um sistema de *help desk* universitário, comparando *Naive Bayes* e SVM (*Support Vector Machine* – Máquina de Vetores de Suporte), um algoritmo que classifica textos encontrando a melhor separação possível entre as categorias, com vetorização TF-IDF. Os resultados demonstraram que o SVM com TF-IDF supera o *Naive Bayes* em acurácia e F1-score (métrica que combina precisão e revocação em um único valor, detalhada na Seção 3.4.1), estabelecendo-se como referência para abordagens clássicas nesse domínio.

Apesar de sua eficiência computacional e facilidade de interpretação, esses modelos apresentam limitações na captura de relações semânticas entre termos. A representação TF-IDF trata cada palavra de forma independente, sem considerar o contexto em que ela aparece, o que restringe a capacidade do modelo de distinguir categorias semanticamente próximas.

Formalmente, o peso de um termo t em um documento d é

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{\text{DF}(t)} \tag{2.1}$$

onde $\text{TF}(t, d)$ é o número de vezes que t aparece em d, N é o total de documentos do corpus e $\text{DF}(t)$ é o número de documentos que contêm t. Por exemplo, em um corpus hipotético de 1.000 chamados, um termo que aparece em 10 deles (como “VPN”) tem $\log_{10}(1000/10) = 2$,

enquanto um que aparece em 500 (como "acesso") tem $\log_{10}(1000/500) \approx 0{,}3$: o termo raro pesa mais.

2.3.3.2 Redes neurais convolucionais aplicadas a *tickets*

Com o avanço das técnicas de aprendizado profundo, modelos baseados em redes neurais convolucionais (CNN) passaram a ser aplicados à classificação de chamados de suporte. Paramesh e Shreedhara (2022) propuseram um classificador baseado em CNN com *word embeddings* (representações vetoriais densas de palavras aprendidas por redes neurais) para categorização automática de *tickets* de *service desk* de infraestrutura de TI. Os autores compararam o modelo proposto com SVM, *Naive Bayes*, Regressão Logística e *K-Nearest Neighbor*, e a CNN obteve a maior acurácia (91,20%) e o maior F1-score (91,02%) no conjunto de teste, contra 87,43% e 87,36% do SVM, o melhor dos modelos clássicos.

A vantagem das CNN em relação aos modelos clássicos reside na capacidade de identificar automaticamente quais grupos de palavras que aparecem juntas são importantes para a classificação, sem que um especialista precise definir isso manualmente. Essa progressão de modelos lineares com TF-IDF para redes profundas com *embeddings* estabelece o contexto histórico sobre o qual os modelos baseados em *transformers* representam um novo salto qualitativo.

2.3.3.3 Classificação de chamados no contexto brasileiro

A literatura sobre classificação automática de chamados de suporte em português brasileiro ainda é escassa, e os trabalhos existentes evidenciam limitações importantes das abordagens mais simples nesse contexto específico.

Pereira et al. (2023) apresentaram um modelo de classificação automática de chamados de suporte de TI em sete categorias, atingindo 75% de precisão média com modelos de aprendizado de máquina tradicionais. Os autores disponibilizaram publicamente o código, o modelo e os dados anonimizados, e desenvolveram um protótipo *web* com *backend* e *frontend* para uso dos analistas de TI, tornando-o um dos poucos trabalhos nacionais com *dataset* público e interface funcional. No entanto, a acurácia de 75% deixa margem considerável para melhoria, especialmente em cenários com maior número de categorias ou com classes semanticamente próximas.

Yokota (2025) investigou a classificação hierárquica de chamados de suporte ao cliente em PT-BR utilizando *Naive Bayes* e MLP (*Multilayer Perceptron* – Perceptron Multicamadas) com duas variações arquiteturais, sobre *embeddings* do modelo Multilingual E5. Os resultados, avaliados por F1-macro para lidar com o desbalanceamento entre classes, foram limitados (no máximo 61%), o que a autora associa ao desbalanceamento, à proximidade semântica entre categorias e ao vocabulário especializado. A autora conclui que a abordagem proposta revelou desafios substanciais na classificação automática de textos

especializados, evidenciando a necessidade de abordagens metodológicas mais sofisticadas, o que motiva avaliar o ajuste fino do BERTimbau neste trabalho.

2.3.4 Representações vetoriais de texto

2.3.4.1 *TF-IDF* e modelos baseados em frequência

A representação mais simples de um documento para fins de classificação é o modelo *bag-of-words*: cada documento é representado como um vetor de contagens de palavras, sem considerar a ordem em que aparecem. O *TF-IDF* é um refinamento desse modelo que pondera cada termo pelo produto de sua frequência no documento e pelo inverso de sua frequência no corpus, reduzindo o peso de termos muito comuns e aumentando o peso de termos mais informativos (RAZMA; JURKUS, 2026).

Apesar de sua simplicidade, o *TF-IDF* ainda é amplamente utilizado como *baseline* em tarefas de classificação de texto, especialmente quando combinado com SVM, e apresenta desempenho competitivo em domínios com vocabulário técnico específico e baixa polissemia, ou seja, onde as palavras raramente mudam de sentido dependendo do contexto (WAHBA, 2023). Sua principal limitação é a incapacidade de capturar relações semânticas entre termos: palavras sinônimas ou semanticamente relacionadas são tratadas como completamente distintas.

2.3.4.2 *Embeddings* estáticos — *Word2Vec* e *GloVe*

Os *embeddings* de palavras representam um avanço significativo em relação às abordagens baseadas em frequência, ao mapear cada palavra para um vetor denso em um espaço de baixa dimensionalidade, de forma que palavras semanticamente relacionadas sejam representadas por vetores próximos. O *Word2Vec*, proposto por Mikolov et al. (2013), aprende *embeddings* a partir de grandes corpora (plural de *corpus*, conjunto amplo de textos usado para treinamento) textuais utilizando redes neurais rasas, treinadas para predizer palavras a partir de seu contexto ou contexto a partir de palavras.

O GloVe (*Global Vectors for Word Representation* – Vetores Globais para Representação de Palavras) utiliza estatísticas globais de co-ocorrência do corpus para produzir representações com propriedades algébricas notáveis: vetores de palavras semanticamente relacionadas tendem a ser próximos no espaço vetorial, e operações aritméticas entre vetores refletem relações semânticas — o exemplo clássico é que o vetor de “rei” menos “homem” mais “mulher” resulta em um vetor próximo ao de “rainha”.

Para tornar a ideia concreta, a Tabela 1 mostra um exemplo ilustrativo com vetores de quatro dimensões para três palavras do vocabulário de TI. Em um modelo real, os vetores têm centenas de dimensões e seus valores são aprendidos a partir do corpus. Na representação *bag-of-words*, “senha” e “login” seriam palavras sem relação alguma

(similaridade 0). Nos *embeddings*, a similaridade do cosseno entre os vetores de "senha" e "login" é alta (0,99), enquanto a de "senha" e "impressora" é baixa (0,12).

Tabela 1 – Exemplo ilustrativo de *embeddings* de quatro dimensões

Palavra	d_1	d_2	d_3	d_4
senha	0,8	0,1	0,7	0,0
login	0,7	0,2	0,6	0,1
impressora	0,0	0,9	0,1	0,8

Par de palavras	**Similaridade do cosseno**
senha – login	0,99
senha – impressora	0,12
login – impressora	0,28

Fonte: Elaborado pelos autores (valores ilustrativos, não aprendidos de um corpus)

Esses *embeddings* estáticos representaram um avanço expressivo na qualidade das representações textuais e foram amplamente adotados como camada de entrada em modelos de aprendizado profundo para PLN. No entanto, eles atribuem um único vetor a cada palavra, independentemente do contexto em que ela aparece, limitação conhecida como o problema da representação estática.

2.3.4.3 Limitações das representações estáticas — polissemia e contexto

O grande problema dos *embeddings* estáticos é que eles não conseguem identificar quando uma palavra muda de sentido dependendo do contexto — como "banco", que pode ser um assento ou uma instituição financeira. Em textos técnicos de suporte de TI, isso se manifesta em termos como "acesso", "serviço" ou "sistema", que podem ter significados muito distintos dependendo do chamado.

Essa limitação motivou o desenvolvimento de representações contextualizadas, nas quais o *embedding* de cada palavra é computado dinamicamente a partir do contexto completo da frase. Os modelos baseados na arquitetura *Transformer*, em particular o BERT, representam a abordagem dominante nessa direção (DEVLIN et al., 2019).

2.3.5 Arquitetura *Transformer* e mecanismo de atenção

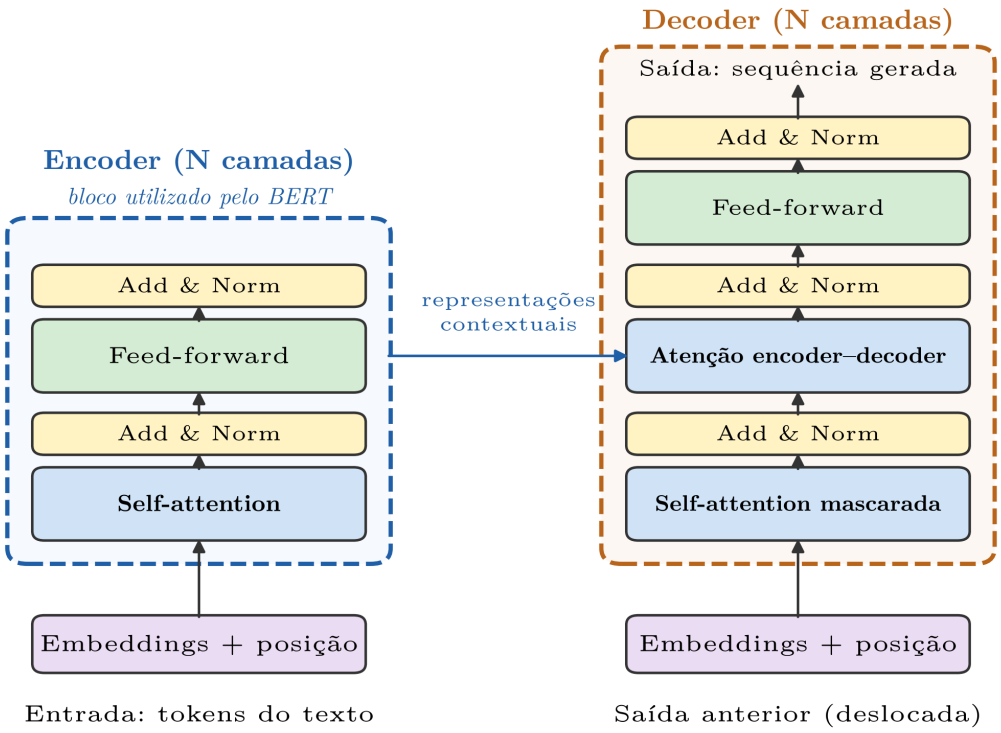
2.3.5.1 *Self-attention* e atenção bidirecional

A arquitetura *Transformer* é composta por dois blocos principais: um *encoder* (codificador) e um *decoder* (decodificador), cada um formado por camadas empilhadas (VASWANI et al., 2017). O *encoder* é responsável por receber a sequência de entrada e transformá-la em representações vetoriais ricas em informação contextual. O *decoder*, por sua vez, utiliza

essas representações para gerar a sequência de saída como uma tradução ou resposta. Cada camada do *encoder* é composta por dois sub-componentes principais: o mecanismo de *self-attention*, que permite que cada *token*, ou seja, cada unidade mínima de texto em que o modelo divide a entrada, "observe" todos os outros da sequência, e uma rede neural densa aplicada posição a posição (*feed-forward network*). O resultado de cada sub-componente passa por uma operação de normalização (*layer normalization*) que estabiliza o treinamento (VASWANI et al., 2017). Vale destacar que modelos como o BERT utilizam apenas o bloco *encoder*, pois seu objetivo é produzir representações contextuais do texto de entrada, e não gerar novas sequências (DEVLIN et al., 2019).

A Figura 1 resume essa estrutura. O BERT utiliza apenas o bloco da esquerda, o *encoder*.

Figura 1 – Arquitetura *Transformer* simplificada



Fonte: Elaborado pelos autores, com base em Vaswani et al. (2017)

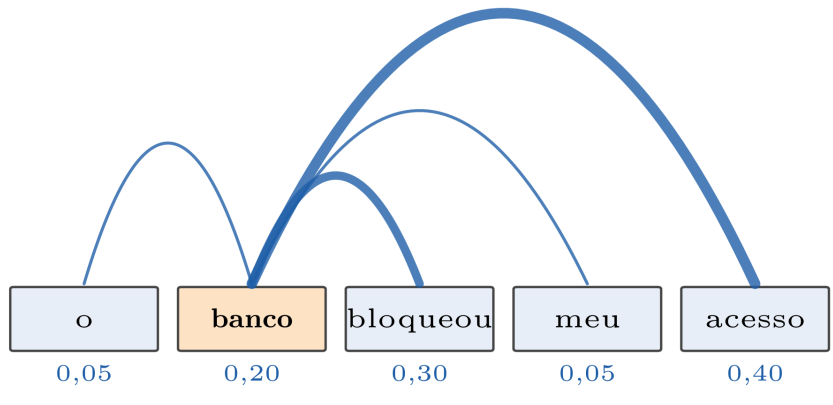
A arquitetura *Transformer*, proposta por Vaswani et al. (2017), introduziu o mecanismo de *self-attention* como alternativa às redes recorrentes para processamento de sequências. As RNN (*Recurrent Neural Network* – Rede Neural Recorrente) são uma classe de redes neurais projetadas para processar dados sequenciais, nos quais o contexto e a ordem temporal importam, processando os *tokens* um por vez, da esquerda para a direita. As LSTM (*Long Short-Term Memory* – Memória de Longo e Curto Prazo) são um tipo especializado de RNN projetado para aprender dependências de longo prazo em sequências, mitigando o problema de desvanecimento do gradiente presente nas RNN simples. Apesar

dessas melhorias, ambas as arquiteturas ainda processam a sequência de forma estritamente sequencial, o que limita o paralelismo computacional e dificulta a captura de relações entre termos distantes.

Para compreender o mecanismo de atenção, considere a frase "o banco bloqueou meu acesso": para determinar o significado da palavra "banco", é necessário considerar as demais palavras da frase — especialmente "bloqueou" e "acesso" — que indicam tratar-se de uma instituição financeira e não de um assento. O *self-attention* formaliza esse processo: para cada *token* da sequência, o mecanismo calcula um score de relevância em relação a todos os outros *tokens*, indicando o quanto cada um deve "influenciar" a representação do *token* atual. "Atender simultaneamente a todas as posições" significa que esse cálculo é realizado em paralelo para todos os pares de *tokens* da sequência, possibilitando a captura de dependências de longo alcance de forma muito mais eficiente do que as arquiteturas recorrentes.

A Figura 2 ilustra esse cálculo para a palavra "banco": os pesos de atenção, que somam 1, indicam o quanto cada *token* contribui para a nova representação de "banco". Nesse exemplo, "acesso" e "bloqueou" recebem os maiores pesos, o que leva o modelo a representar "banco" como instituição financeira.

Figura 2 – Exemplo ilustrativo de *self-attention* para o *token* "banco"

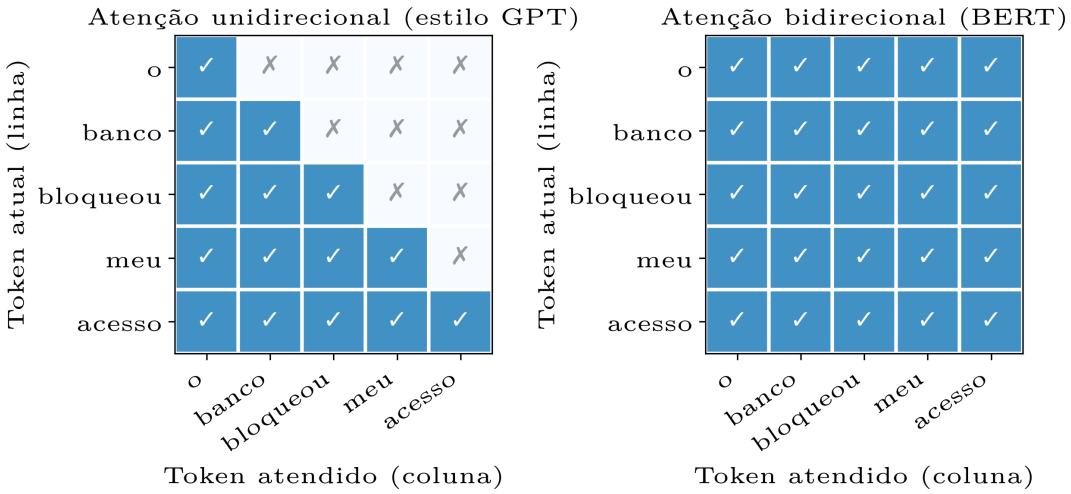


Fonte: Elaborado pelos autores (valores ilustrativos, não extraídos de um modelo treinado)

Modelos como o GPT utilizam atenção unidirecional (cada *token* só pode atender aos *tokens* anteriores). O BERT, por sua vez, utiliza atenção bidirecional: cada *token* atende simultaneamente a todos os outros *tokens* da sequência, tanto os que aparecem antes quanto os que aparecem depois. Essa bidirecionalidade é fundamental para a qualidade das representações contextualizadas geradas pelo modelo, uma vez que o significado de uma palavra frequentemente depende tanto de seu contexto anterior quanto posterior (DEVLIN et al., 2019).

A Figura 3 compara os dois tipos de atenção: na atenção unidirecional, cada *token* só atende a si mesmo e aos anteriores; na bidirecional, atende a todos.

Figura 3 – Atenção unidirecional e atenção bidirecional



Fonte: Elaborado pelos autores. Cada linha indica a quais *tokens* a palavra daquela linha pode atender

2.3.5.2 Pré-treinamento — *Masked Language Modeling* e *Next Sentence Prediction*

O BERT é pré-treinado em dois objetivos complementares sobre grandes corpora textuais não rotulados. O primeiro, MLM (*Masked Language Modeling* – Modelagem de Linguagem com Máscara), mascara aleatoriamente 15% dos *tokens* de cada sequência de entrada e treina o modelo para predizer os *tokens* mascarados a partir do contexto bidirecional restante. Esse objetivo força o modelo a aprender representações ricas que capturam o significado de cada palavra em seu contexto completo (DEVLIN et al., 2019).

O segundo objetivo, NSP (*Next Sentence Prediction* – Predição da Próxima Sentença), treina o modelo a determinar se duas sentenças apresentadas em sequência são consecutivas no texto original ou foram pareadas aleatoriamente. Esse objetivo permite ao modelo aprender relações semânticas entre sentenças, o que é útil para tarefas como inferência textual e resposta a perguntas.

2.3.5.3 *Fine-tuning* para tarefas *downstream*

Após o pré-treinamento, o BERT pode ser adaptado para tarefas específicas por meio do processo de *fine-tuning*: o modelo pré-treinado é inicializado com os pesos aprendidos no pré-treinamento, uma camada de saída específica para a tarefa é adicionada ao topo da arquitetura, e o modelo completo é treinado por um número reduzido de épocas sobre o conjunto de dados rotulado da tarefa-alvo (DEVLIN et al., 2019).

Essa abordagem de pré-treinamento seguido de *fine-tuning* é particularmente eficaz porque o modelo já adquiriu conhecimento linguístico geral durante o pré-treinamento,

e o *fine-tuning* apenas adapta esse conhecimento para o domínio e a tarefa específicos. Para classificação de chamados de TI, o *fine-tuning* ajusta os pesos do modelo para que as representações produzidas sejam especialmente discriminativas entre as categorias de chamados presentes no *dataset*.

2.3.6 BERT

2.3.6.1 Arquitetura e configurações *BERT-base* e *BERT-large*

Devlin et al. (2019) propuseram o BERT em duas configurações. O *BERT-base* possui 12 camadas de *encoder Transformer*, 12 cabeças de atenção e 110 milhões de parâmetros. O *BERT-large* possui 24 camadas, 16 cabeças de atenção e 340 milhões de parâmetros. Ambas as versões foram pré-treinadas sobre o corpus da Wikipedia em inglês e o BookCorpus, totalizando aproximadamente 3,3 bilhões de palavras.

O BERT estabeleceu o estado da arte em onze tarefas de PLN no momento de sua publicação, incluindo classificação de texto, reconhecimento de entidades nomeadas, resposta a perguntas e inferência textual. Seu lançamento popularizou o uso de modelos pré-treinados em tarefas de PLN, consolidando a abordagem de pré-treinamento em larga escala seguido de *fine-tuning* como a estratégia dominante para tarefas de PLN.

2.3.6.2 *Token* [CLS] e classificação de sequências

Para tarefas de classificação de sequências, o BERT utiliza um *token* especial [CLS] (abreviação de *Classification*, marcador inserido pelo modelo para agregar a representação global da sequência) inserido no início de cada sequência de entrada. Após o processamento pelas camadas de *encoder*, a representação vetorial correspondente ao [CLS] agrega informação contextual de toda a sequência e é usada como entrada para a camada de classificação (DEVLIN et al., 2019). Durante o *fine-tuning*, essa camada de classificação, tipicamente uma camada *linear* seguida de *softmax*, aprende a mapear a representação do [CLS] para as probabilidades de cada categoria. Neste trabalho, essa representação é usada para classificar o chamado de TI em uma das categorias predefinidas, e os valores SHAP e LIME são calculados sobre os tokens de entrada para identificar quais contribuíram mais para a predição.

2.3.7 BERTimbau — BERT para o português brasileiro

2.3.7.1 Corpus de pré-treinamento e diferenças em relação ao mBERT

Souza, Nogueira e Lotufo (2020) desenvolveram o BERTimbau, modelo BERT pré-treinado especificamente para o português brasileiro, a partir do BrWaC (*Brazilian Web as Corpus*), um grande corpus de páginas *web* em português com 2,68 bilhões de *tokens*.

Duas versões foram disponibilizadas: BERTimbau-base (12 camadas, 110M parâmetros) e BERTimbau-large (24 camadas, 335M parâmetros).

A principal diferença em relação ao mBERT está no foco linguístico: o mBERT foi pré-treinado sobre 104 idiomas simultaneamente, o que dilui a capacidade de representação para cada língua individualmente. O BERTimbau, ao ser pré-treinado exclusivamente sobre texto em português brasileiro, aprende padrões morfológicos, sintáticos e semânticos específicos do idioma com maior profundidade.

2.3.7.2 Desempenho em tarefas de PLN em PT-BR

Souza, Nogueira e Lotufo (2020) avaliaram o BERTimbau em três tarefas de PLN em português: reconhecimento de entidades nomeadas (NER), similaridade textual entre sentenças e inferência textual (*Recognizing Textual Entailment*). O modelo obteve desempenho de estado da arte nas três tarefas, consolidando-se como referência para aplicações de PLN em PT-BR.

Para o presente trabalho, a escolha do BERTimbau em detrimento do mBERT é motivada por dois fatores. Primeiro, os chamados de suporte analisados estão em português brasileiro, idioma para o qual o BERTimbau possui representações mais ricas. Segundo, os chamados utilizam terminologia técnica de TI em português, que pode diferir substancialmente de sua versão em inglês. E o BERTimbau, pré-treinado em corpus nacional, tem maior probabilidade de ter encontrado esse vocabulário durante o pré-treinamento.

O BERTimbau foi mantido como modelo deste trabalho, dado que os chamados analisados estão predominantemente em português brasileiro. A avaliação de modelos multilíngues ou mais recentes fica indicada como trabalho futuro.

2.3.8 *Fine-tuning* de BERT para classificação de chamados de TI

A aplicação de BERT e modelos derivados à classificação de chamados de TI tem demonstrado resultados expressivos na literatura recente. Borges (2024) realizou uma comparação sistemática entre BERT e algoritmos clássicos, SVM e *Naive Bayes*, em tarefas de classificação de texto em português e inglês, utilizando quatro *datasets* públicos. Os resultados confirmaram que o BERT supera consistentemente os modelos tradicionais em acurácia e F1, especialmente em tarefas multiclasse com nuances semânticas mais complexas, embora com custo computacional consideravelmente maior.

Zemp (2021) comparou CNN, BiLSTM e RoBERTa sobre mais de um milhão de *tickets* reais de um banco privado de médio porte, em mais de 300 classes. A melhor CNN atingiu F1-macro de 0,75. Com o *fine-tuning* simples, o RoBERTa ficou abaixo desse valor; somente após um pré-treinamento adicional do modelo de linguagem com os próprios *tickets* ele superou a CNN, por uma margem de 0,018.

Razma e Jurkus (2026) compararam regressão logística com TF-IDF frente a mBERT e XLM-RoBERTa, uma variante do RoBERTa desenvolvida especificamente para lidar com múltiplos idiomas simultaneamente, em um corpus de 87 mil *tickets* multilíngues de uma empresa de manufatura. Os resultados confirmaram que *transformers* superam modelos clássicos em macro-F1, especialmente para classes de baixa frequência e semanticamente complexas. Os autores aplicaram o LIME ao mBERT e à regressão logística, e o SHAP à regressão logística, o que mostra ser viável explicar um BERT *fine-tuned* com LIME sobre chamados reais de ITSM; eles afirmam que as explicações contribuem para a interpretabilidade e a confiança.

2.3.9 Definição e motivação

2.3.9.1 O problema da caixa-preta em aprendizado profundo

Modelos de aprendizado profundo, incluindo o BERT, são frequentemente descritos como caixas-pretas: produzem predições com alta acurácia, mas sem revelar quais características da entrada determinaram a saída. Isso acontece porque esses modelos possuem centenas de milhões de parâmetros internos, valores numéricos ajustados durante o treinamento, organizados em múltiplas camadas de cálculos complexos. Essa estrutura torna impossível para um ser humano acompanhar ou explicar, passo a passo, como o modelo chegou a uma determinada decisão (SALIH et al., 2023).

Em contextos como a triagem de chamados de TI, essa opacidade representa um problema prático concreto. Analistas de suporte e gestores precisam auditar as decisões do sistema, identificar erros sistemáticos e eventualmente contestar categorizações incorretas. Um modelo que não fornece justificativa para suas predições é difícil de depurar, difícil de melhorar e difícil de justificar perante *stakeholders* (partes interessadas, como gestores e diretores) organizacionais.

2.3.9.2 Confiança, explicabilidade e adoção em produção

No estudo de Zemp (2021), a equipe de suporte recusou parte do sistema de classificação, como discutido na Seção 2.4.5.2. Razma e Jurkus (2026) afirmam que as explicações LIME favorecem a interpretabilidade e a confiança no sistema, pois permitem verificar se os termos destacados como mais influentes fazem sentido para aquela categoria específica, embora não tenham medido a reação dos operadores.

Essa ideia é consistente com a literatura de XAI mais ampla: as explicações ajudam os usuários a decidir se confiam em uma predição e a identificar modelos que erram por razões inadequadas (RIBEIRO; SINGH; GUESTRIN, 2016).

2.3.9.3 Taxonomia de métodos XAI — intrínsecos versus *post-hoc*, locais versus globais

Os métodos de XAI podem ser classificados segundo duas dimensões principais (SALIH et al., 2023). A primeira dimensão distingue métodos intrínsecos dos métodos *post-hoc*. Métodos intrínsecos são aqueles em que a interpretabilidade é uma propriedade do próprio modelo — árvores de decisão, regressão logística e modelos lineares em geral são interpretáveis por *design*, pois seus parâmetros têm significado diretamente interpretável. Métodos *post-hoc* são aplicados após o treinamento, sobre modelos de qualquer complexidade, e produzem explicações sem modificar o modelo original.

A segunda dimensão distingue explicações locais das globais. Explicações locais descrevem o comportamento do modelo para uma instância específica de entrada (exemplo: por que o modelo classificou *este* chamado como "Acesso"?). Explicações globais descrevem o comportamento geral do modelo (exemplo: quais termos são mais importantes para cada categoria em média?). SHAP e LIME são métodos *post-hoc* que operam primariamente no nível local, embora o SHAP permita agregações que produzem explanações globais.

2.3.10 LIME — *Local Interpretable Model-Agnostic Explanations*

2.3.10.1 Funcionamento — perturbações e modelo local interpretável

LIME foi proposto por Ribeiro, Singh e Guestrin (2016) como um método de explicação local que funciona com qualquer tipo de modelo e qualquer tipo de entrada, sem precisar conhecer sua estrutura interna. O método parte do princípio de que, embora um modelo de aprendizado profundo seja globalmente complexo e não-linear, seu comportamento em torno de um exemplo específico pode ser aproximado por um modelo simples e interpretável.

O processo de geração de uma explicação LIME para uma instância de texto ocorre em quatro etapas. Primeiro, o texto original é perturbado pela remoção ou substituição aleatória de palavras, criando um conjunto de versões modificadas da frase. Segundo, cada versão modificada é submetida ao modelo para que ele produza uma predição, permitindo observar se e como a resposta muda conforme as palavras são alteradas. Terceiro, as versões modificadas são ponderadas pela similaridade com o texto original, sendo que versões mais próximas do original recebem peso maior. Quarto, um modelo linear simples é ajustado sobre esse conjunto, e seus coeficientes indicam quais palavras mais influenciaram a classificação (RIBEIRO; SINGH; GUESTRIN, 2016).

Formalmente, o LIME resolve o seguinte problema de otimização para encontrar a explicação $\xi(x)$ de uma instância x:

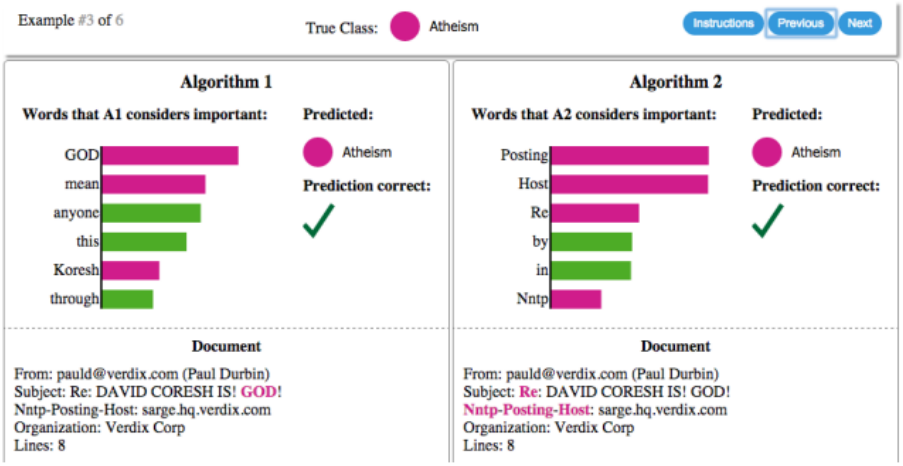
$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{2.2}$$

onde f é o modelo de caixa-preta a ser explicado, G é o conjunto de modelos interpretáveis candidatos, $\mathcal{L}$ é uma função de perda que mede a fidelidade da explicação g ao

modelo f na vizinhança de x, π_x é uma função de proximidade que pondera as instâncias perturbadas pela similaridade com x, e $\Omega(g)$ é uma medida de complexidade do modelo explicativo (RIBEIRO; SINGH; GUESTRIN, 2016). O objetivo é encontrar o modelo interpretável mais simples possível que seja localmente fiel ao comportamento do modelo complexo.

A Figura 4 ilustra esse processo: as palavras destacadas indicam sua contribuição positiva ou negativa para a classe predita.

Figura 4 – Exemplo de explicação gerada pelo LIME



Fonte: Ribeiro, Singh e Guestrin (2016)

2.3.10.2 Aplicação em classificação de texto

No contexto de classificação de texto, cada palavra (ou *token*) da entrada é tratada como uma *feature*, ou seja, uma característica binária que indica se está presente ou ausente no texto. A explicação LIME indica quais palavras, quando presentes, aumentam a probabilidade de cada classe, e quais palavras a diminuem. O resultado pode ser visualizado como um destaque colorido sobre o texto original, onde palavras em verde contribuem positivamente para a classe predita e palavras em vermelho contribuem negativamente (MUSTAFA; Hama Saeed, 2025).

Mustafa e Hama Saeed (2025) demonstraram que a aplicação de LIME na classificação de texto não apenas produz explicações úteis para o usuário final, mas também contribui para melhorar o processo de pré-processamento: ao revelar que *stopwords*, palavras muito frequentes e de pouco conteúdo informativo, como artigos e preposições (“de”, “o”, “para”), recebem pesos desproporcionalmente altos nas explicações, o LIME evidenciou a necessidade de sua remoção.

2.3.10.3 Vantagens e limitações

A principal vantagem do LIME é sua generalidade: o método funciona com qualquer modelo e qualquer tipo de entrada sem necessidade de acesso ao interior do modelo. Isso o torna especialmente adequado para modelos de caixa-preta como o BERT, cujos mecanismos internos são de difícil interpretação direta.

Sua principal limitação é a instabilidade das explicações. Por depender de amostragem aleatória de perturbações, execuções diferentes do LIME sobre a mesma instância podem produzir explicações distintas, especialmente quando há colinearidade entre as *features*, ou seja, quando algumas características extraídas do texto são muito parecidas ou carregam informações semelhantes (SALIH et al., 2023). Para mitigar essa instabilidade, recomenda-se executar o LIME múltiplas vezes e analisar a consistência das explicações, ou fixar a semente aleatória para garantir reprodutibilidade.

2.3.11 SHAP — *SHapley Additive exPlanations*

2.3.11.1 Fundamentos — valores de Shapley da teoria dos jogos

SHAP foi proposto por Lundberg e Lee (2017) com base nos valores de Shapley da teoria dos jogos cooperativos. O problema que o SHAP resolve é: dada uma predição específica de um modelo, como distribuir o "crédito" por essa predição de forma justa entre as *features* de entrada?

Na teoria dos jogos, o valor de Shapley de um jogador em um jogo cooperativo é calculado como a média ponderada de quanto cada jogador contribuiu individualmente em todas as combinações possíveis com os demais jogadores. Lundberg e Lee (2017) adaptam esse conceito para o contexto de modelos de aprendizado de máquina: cada *feature* é um "jogador", a predição do modelo é o "valor do jogo", e o valor SHAP de cada *feature* mede o quanto ela contribuiu para a predição, considerando todas as possíveis ordens em que as *features* poderiam ser incluídas.

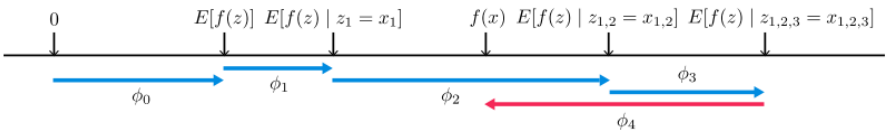
Formalmente, o valor de Shapley de uma *feature* i é calculado como:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S) \right] \tag{2.3}$$

onde F é o conjunto de todas as *features*, S é um subconjunto que não inclui i, $f(S)$ é a predição do modelo usando apenas as *features* em S, e $f(S \cup \{i\})$ é a predição incluindo a *feature* i (LUNDBERG; LEE, 2017). O fator $\frac{|S|!(|F|-|S|-1)!}{|F|!}$ garante que todas as ordenações possíveis de inclusão das *features* sejam consideradas com peso igual, assegurando a propriedade de simetria.

A Figura 5 ilustra esse conceito: cada *feature* desloca a predição do valor base até o valor final do modelo.

Figura 5 – Representação dos valores SHAP por *feature*



Fonte: Lundberg e Lee (2017)

2.3.11.2 Propriedades matemáticas — eficiência, simetria e ausência

A formulação baseada em valores de Shapley garante ao SHAP propriedades matemáticas desejáveis, que o distinguem de métodos *ad hoc* de atribuição de importância. Lundberg e Lee (2017) as formulam como acurácia local, ausência (*missingness*) e consistência. Na forma clássica dos axiomas de Shapley, as propriedades de interesse aqui são três. A propriedade de eficiência estabelece que a soma dos valores SHAP de todas as *features* iguala a diferença entre a predição do modelo para a instância em questão e o valor base (predição média do modelo sobre o conjunto de dados). A propriedade de simetria estabelece que *features* com contribuições iguais ao modelo recebem valores SHAP iguais, independentemente de sua posição na entrada. A propriedade de ausência estabelece que *features* que não afetam a predição recebem valor SHAP zero.

Essas propriedades conferem ao SHAP uma fundamentação teórica sólida que o LIME não possui, tornando suas explicações mais consistentes e matematicamente justificáveis.

2.3.11.3 Cálculo do SHAP em modelos de texto

Calcular o valor exato de Shapley se torna inviável quando o modelo possui muitas *features*, pois seria necessário testar todas as combinações possíveis entre elas, o que cresce exponencialmente. Para resolver isso, o *KernelSHAP* (LUNDBERG; LEE, 2017) estima esses valores testando apenas uma amostra dessas combinações, tornando o método viável mesmo em modelos de texto, onde cada *token* é tratado como uma *feature*.

Neste trabalho, o SHAP foi calculado com a biblioteca *shap*, tratando cada palavra distinta do campo como uma *feature* e a probabilidade da classe predita como o valor a ser explicado. Nos campos com até 10 palavras distintas, o valor de Shapley foi calculado de forma exata, testando todas as combinações; acima disso, foi estimado por amostragem de permutações, e não pelo *KernelSHAP*. O resultado é um valor por palavra: valores SHAP positivos contribuem para a classe predita e valores negativos contribuem contra ela. Esses valores são utilizados como pesos dos nós no grafo de co-ocorrência, em conjunto com os valores LIME, permitindo comparação visual entre os dois métodos de explicabilidade.

2.3.12 Comparação entre SHAP e LIME

2.3.12.1 Consistência e estabilidade das explicações

Salih et al. (2023) realizaram uma análise comparativa abrangente entre SHAP e LIME, avaliando suas propriedades teóricas e comportamento empírico em tarefas de classificação. Em termos de consistência, o SHAP demonstra maior estabilidade: por ser baseado em uma formulação matemática única com garantias teóricas, suas explicações são menos sensíveis à aleatoriedade do processo de amostragem. O LIME, por depender de perturbações aleatórias e de um processo de ajuste local, pode produzir explicações distintas para a mesma instância em execuções diferentes.

Os autores concluem que SHAP é preferível quando a consistência e a fundamentação teórica das explicações são prioritárias, enquanto o LIME pode ser preferível quando a velocidade de geração das explicações é mais importante do que a estabilidade.

2.3.12.2 Custo computacional

Em termos de custo computacional, o LIME é mais eficiente que o *KernelSHAP* em textos longos porque trabalha com um número fixo de perturbações. Já o *KernelSHAP* exige uma amostragem muito maior de combinações de *features* para gerar estimativas precisas, o que eleva o tempo de processamento (SALIH et al., 2023). Para textos curtos, como os chamados de suporte de TI, que tipicamente contêm poucas dezenas de palavras, essa diferença de custo é menos pronunciada.

2.3.12.3 Justificativa para uso conjunto neste trabalho

A decisão de implementar e comparar SHAP e LIME neste trabalho, em vez de escolher apenas um dos métodos, é fundamentada em duas razões. Primeiro, a comparação entre os dois métodos produz evidência empírica sobre a qualidade e consistência das explicações geradas para chamados de TI em PT-BR, contribuição metodológica relevante para a literatura sobre XAI em domínios especializados. Segundo, a construção do grafo de co-ocorrência proposto utiliza os valores SHAP e LIME como pesos dos nós, permitindo avaliar se a escolha do método de XAI afeta a estrutura do grafo resultante.

2.3.13 XAI aplicado a classificação de texto e tickets de TI

2.3.13.1 LIME em classificação multiclasse de texto

Mustafa e Hama Saeed (2025) propuseram um *framework* de classificação de texto que combina PLN e XAI. O sistema inclui pré-processamento com NLTK (*Natural Language Toolkit* – Kit de Ferramentas de Linguagem Natural), balanceamento de classes com SMOTE (*Synthetic Minority Oversampling Technique* – Técnica de Sobreamostragem Sintética de Minorias), técnica que cria exemplos sintéticos das classes minoritárias a

partir da interpolação entre exemplos vizinhos, para lidar com *datasets* desbalanceados, e validação cruzada estratificada repetida: uma técnica que divide os dados em múltiplos subconjuntos para avaliar o modelo de forma mais robusta, garantindo que todas as classes estejam representadas em cada divisão. O modelo utilizado foi um híbrido de regressão *ridge* com regressão logística. O LIME foi utilizado para validar a explicabilidade das classificações em seis domínios distintos: medicina, esportes, entretenimento, política, tecnologia e negócios. O modelo híbrido atingiu 99% de acurácia após a aplicação do pré-processamento informado pelas explanações LIME.

O trabalho demonstra que a integração de XAI ao *pipeline* de classificação não é apenas uma camada de interpretabilidade adicionada ao final do processo, mas pode também influenciar decisões de pré-processamento e seleção de *features*, melhorando a qualidade do modelo como um todo.

2.3.13.2 SHAP e LIME em ambiente ITSM corporativo

Razma e Jurkus (2026) aplicaram o LIME à regressão logística com *TF-IDF* e ao mBERT *fine-tuned*, e o SHAP à regressão logística, em um ambiente ITSM corporativo real. Os resultados qualitativos das explicações foram significativamente distintos: o modelo linear tendia a destacar palavras-chave literais com alta frequência no corpus, enquanto o BERT destacava termos semanticamente coerentes com o significado do chamado.

Essa diferença qualitativa nas explicações, documentada por Razma e Jurkus (2026), sugere que o BERTimbau, combinado com SHAP e LIME, tende a produzir explicações mais acuradas em termos de importância de *features* e potencialmente mais interpretáveis para analistas de TI sem formação em aprendizado de máquina, hipótese que este trabalho verifica empiricamente, avaliando se os termos destacados fazem sentido para cada categoria de chamado classificada.

2.3.14 Fundamentos de grafos aplicados a texto

2.3.14.1 Definição de grafo de co-ocorrência — nós, arestas e janela deslizante

Um grafo de co-ocorrência de texto é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, onde:

- V representa o conjunto de nós, correspondendo aos *tokens* únicos do texto;
- E representa o conjunto de arestas, conectando pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho w;
- W representa os pesos das arestas, determinados pela frequência de co-ocorrência entre os *tokens*;
- f_{shap} representa os pesos dos nós calculados pelo método SHAP, indicando a contribuição de cada *token* para a predição do modelo (LUNDBERG; LEE, 2017);

- f_{lime} representa os pesos dos nós calculados pelo método LIME, permitindo comparação visual entre os dois métodos de explicabilidade (RIBEIRO; SINGH; GUESTRIN, 2016).

A janela deslizante percorre o texto posição por posição; em cada posição, todos os pares de *tokens* dentro da janela recebem uma aresta ou têm o peso de sua aresta existente incrementado (YAO; MAO; LUO, 2019). Essa representação captura informação estrutural sobre o texto que as representações vetoriais não preservam: dois *tokens* que frequentemente aparecem juntos no mesmo contexto estarão conectados por uma aresta de alto peso no grafo, independentemente de sua posição absoluta na sequência. Para chamados de TI, isso significa que termos técnicos que tendem a co-ocorrer em chamados de uma mesma categoria, como "VPN" e "acesso" em chamados de conectividade, estarão estruturalmente próximos no grafo, com pesos de nós proporcionais à sua importância explicativa calculada por SHAP e LIME.

2.3.14.2 Ponderação de arestas — frequência, PMI

O peso das arestas em um grafo de co-ocorrência pode ser determinado por diferentes métricas. A abordagem mais simples utiliza a frequência bruta de co-ocorrência, ou seja, a contagem de quantas vezes o par de *tokens* co-ocorreu dentro da janela deslizante. Métricas mais sofisticadas, como a PMI (*Pointwise Mutual Information* – Informação Mútua Pontual), normalizam a frequência de co-ocorrência pelas frequências individuais de cada *token*, favorecendo pares que co-ocorrem mais do que seria esperado pelo acaso (YAO; MAO; LUO, 2019).

A PMI de um par de *tokens* a e b é

$$\text{PMI}(a, b) = \log \frac{p(a, b)}{p(a)\, p(b)} \tag{2.4}$$

onde $p(a, b)$ é a fração de janelas em que a e b aparecem juntos, e $p(a)$ e $p(b)$ são as frações de janelas em que cada um aparece. A frequência de co-ocorrência é, portanto, normalizada pelas frequências individuais dos *tokens*: um par só recebe PMI alta se aparecer junto mais do que o esperado se os termos fossem independentes.

No presente trabalho, os pesos das arestas foram determinados pelo número de co-ocorrências do par dentro da janela deslizante (frequência bruta, sem normalização), enquanto os pesos dos *nós*, que constituem o elemento central da proposta, foram determinados pelos valores SHAP e LIME de cada *token*, calculados sobre as predições do BERTimbau, o que permite comparar visualmente a importância atribuída por cada método. Essa distinção entre ponderação de arestas (estrutura relacional) e ponderação de nós (relevância explicativa) é o diferencial da camada de visualização proposta.

2.3.15 *TextGCN* — formalização do grafo de co-ocorrência

2.3.15.1 Grafo heterogêneo documento-palavra

Yao, Mao e Luo (2019) propuseram o *TextGCN* (*Text Graph Convolutional Network*), modelo que constrói um único grafo heterogêneo para todo o corpus, conectando documentos e palavras. As arestas documento-palavra recebem pesos baseados em TF-IDF, refletindo a importância de cada palavra para cada documento. As arestas palavra-palavra recebem pesos baseados em PMI calculada sobre co-ocorrências em janela deslizante de tamanho 20 sobre todo o corpus, refletindo a força de associação semântica entre pares de palavras.

Essa estrutura permite que as GCN (*Graph Convolutional Network* – Rede Neural Convolucional em Grafos) propaguem informação simultaneamente entre documentos e palavras, capturando relações semânticas globais que modelos baseados em sequências não conseguem representar diretamente.

2.3.15.2 Redes neurais de grafos e propagação de informação

As GNN (*Graph Neural Network* – Rede Neural de Grafos) operam sobre grafos propagando informação entre nós vizinhos por meio de operações de agregação e transformação. Em cada camada da rede, a representação de cada nó é atualizada com base nas representações de seus vizinhos na iteração anterior (YAO; MAO; LUO, 2019). Após múltiplas camadas, cada nó possui uma representação que agrega informação de sua vizinhança local de múltiplos saltos.

No contexto deste trabalho, a relevância do *TextGCN* é fundamentalmente teórica: ele formaliza matematicamente a estrutura do grafo de co-ocorrência e demonstra sua utilidade para classificação de texto. O presente trabalho não implementou GCN: o grafo de co-ocorrência foi utilizado exclusivamente como estrutura de visualização das explicações XAI, e não como componente do modelo de classificação.

2.3.16 Combinação de BERT com grafos de co-ocorrência

2.3.16.1 *Embeddings* BERT como *features* iniciais dos nós

Yang e Cui (2021) propuseram o BEGNN (*BERT-Enhanced Graph Neural Network* – Rede Neural de Grafos Aprimorada por BERT), que combina representações contextualizadas do BERT com grafos de co-ocorrência para classificação de texto. Na arquitetura BEGNN, o BERT gera *embeddings* contextualizados para cada *token* da entrada; esses *embeddings* são utilizados como *features* iniciais dos nós do grafo de co-ocorrência correspondente ao documento. Em seguida, uma GCN propaga informação entre nós vizinhos, produzindo representações enriquecidas pela estrutura do grafo. A representação final do documento é obtida por agregação das representações dos nós.

Essa combinação aborda uma limitação do BERT quando utilizado isoladamente: o mecanismo de atenção do BERT captura relações entre *tokens*, mas essas relações são implícitas nos pesos de atenção e não explicitamente estruturadas como um grafo. O grafo de co-ocorrência fornece uma estrutura explícita que complementa as representações contextualizadas.

2.3.16.2 Resultados em vocabulário técnico especializado

Yang e Cui (2021) avaliaram o BEGNN em múltiplos *datasets* de classificação de texto, demonstrando que a combinação BERT + grafo supera tanto o BERT isolado quanto o *TextGCN*, especialmente em *datasets* com vocabulário técnico especializado. Esse resultado é relevante para o presente trabalho porque chamados de suporte de TI são exatamente o tipo de texto com vocabulário técnico especializado para o qual o BEGNN demonstrou vantagem, sugerindo que a estrutura de grafo de co-ocorrência captura informação complementar às representações do BERTimbau.

2.3.17 Grafo de co-ocorrência combinado com XAI

2.3.17.1 Ponderação de nós por *scores* de explicabilidade

Talebi e Abdolvand (2025) apresentam o trabalho de maior proximidade metodológica com o presente TCC: aplicam RoBERTa combinado com LIME e um grafo de co-ocorrência para detectar *body shaming*, prática de humilhar pessoas com base em sua aparência física, em redes sociais em inglês. No trabalho de Talebi e Abdolvand (2025), o grafo de co-ocorrência é construído com nós representando *tokens* e arestas ponderadas pelos *scores* LIME de cada *token*. Os *tokens* com maior importância LIME produzem arestas de maior peso com seus vizinhos na janela de co-ocorrência.

O presente trabalho adotou uma abordagem estruturalmente análoga, mas com diferenças importantes: utilizou o BERTimbau em vez do RoBERTa, aplicou os valores SHAP e LIME como pesos dos nós, em vez dos *scores* LIME como pesos de arestas, e o domínio de aplicação foi a classificação de chamados de suporte de TI em português brasileiro. A escolha de ponderar os nós em vez das arestas reflete a intenção de destacar visualmente os *tokens* individualmente mais relevantes para a predição, enquanto as arestas mantêm sua função de representar a estrutura de co-ocorrência do texto.

2.3.17.2 Visualização interativa como camada de auditabilidade

A literatura aponta que a combinação de grafos de co-ocorrência com *scores* de explicabilidade pode ser exibida em interfaces interativas que tornam o processo decisório auditável. Nessa abordagem, os nós do grafo são dimensionados proporcionalmente à importância de cada *token* calculada pelo método XAI, e as arestas são ponderadas pela

frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo (TALEBI; ABDOLVAND, 2025).

Essa camada de visualização tem o potencial de transformar a saída opaca do modelo em uma representação auditável, o que responde à limitação apontada por Zemp (2021) e Razma e Jurkus (2026): sem acesso às razões da decisão, analistas de suporte não conseguem verificar o comportamento do modelo. O presente trabalho avalia a propriedade técnica de auditabilidade da representação proposta, sem medir a adoção efetiva do sistema por usuários. A solução proposta adota essa abordagem, cuja implementação é detalhada no Capítulo 4.

2.4 Trabalhos correlatos e limitações

Os quatro trabalhos a seguir enfrentaram o desafio de automatizar a triagem de chamados de suporte, servindo de base comparativa para a evolução proposta neste TCC.

2.4.1 Trabalho 1: Text Classification of Service Desk Tickets

Zemp (2021) abordou o problema da triagem automática num ambiente corporativo com mais de um milhão de chamados. Para resolvê-lo, o autor comparou modelos CNN, BiLSTM e RoBERTa. A metodologia envolveu o treino em larga escala num *dataset* real de um banco privado de médio porte. Os resultados mostraram uma acurácia de 92%. No entanto, a classificação automática entre incidentes e requisições foi considerada potencialmente perigosa pela equipe de suporte, devido ao risco de avaliações superficiais dos chamados. O presente trabalho diferencia-se por focar na explicabilidade via SHAP, LIME e grafos, permitindo que os analistas de TI entendam o "porquê" da classificação.

2.4.2 Trabalho 2: A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets)

Wahba (2023) enfrentou o mesmo desafio de classificação de *tickets*, mas propôs uma abordagem focada em eficiência para domínios especializados. A solução utilizou um modelo híbrido (HOOM) que combina SVM linear e regras definidas com especialistas a um classificador *online*. A metodologia demonstrou que em textos técnicos, modelos mais simples podem ser competitivos: a autora testou BERT e XLM e não encontrou ganho significativo sobre o SVM linear. Os resultados mostraram boa performance com baixo custo computacional. A limitação, em relação a este trabalho, é que a tese usa dados em inglês, não públicos, e não avalia explicações por palavra com métodos como SHAP e LIME. O presente trabalho adotou o BERTimbau, específico para PT-BR, que captura

o contexto das palavras, e o combinou com SHAP, LIME e visualização em grafos para explicar as classificações.

2.4.3 Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chamados de suporte

Yokota (2025) investigou a classificação de chamados de atendimento ao cliente especificamente em português brasileiro, utilizando *Naive Bayes* e MLP sobre *embeddings* do Multilingual E5. A metodologia focou na estrutura hierárquica das categorias (31 categorias e 240 subcategorias). Os resultados apresentaram F1-macro limitado, no máximo 61%. A lacuna principal foi o desempenho baixo dos classificadores, que a autora associa ao desbalanceamento, à proximidade semântica entre categorias e ao vocabulário especializado, sem tratar da explicabilidade das decisões. O presente trabalho diferencia-se pelo ajuste fino de uma arquitetura de estado da arte (BERTimbau) pré-treinada em português e pela explicação das decisões com SHAP, LIME e grafos.

2.4.4 Trabalho 4: AI-Based Classification of IT Support Requests in Enterprise Service Management Systems

Razma e Jurkus (2026) compararam regressão logística com TF-IDF, mBERT e XLM-RoBERTa na classificação de mais de 87 mil chamados multilíngues de uma empresa de manufatura, e aplicaram o LIME (à regressão logística e ao mBERT) e o SHAP (à regressão logística) a exemplos de chamados. Os *transformers* tiveram F1-macro maior, e as explicações do BERT destacaram termos mais contextuais que as do modelo linear. Pelo que o artigo apresenta, as explicações são mostradas em exemplos, sem medidas de fidelidade, estabilidade ou concordância entre os métodos, e sem estruturá-las em grafo. O presente trabalho mede essas propriedades para SHAP e LIME sobre o BERTimbau, em chamados em português, e as representa em um grafo de co-ocorrência.

Tabela 2 – Comparação entre trabalhos relacionados

Critério	**Zemp (2021)**	**Wahba (2023)**	**Yokota (2025)**	**Razma e Jurkus (2026)**	**Este trabalho**
Objetivo	Classificação	Eficiência	Classificação PT-BR	Classificação + XAI	Classificação + XAI
Tecnologia	RoBERTa / CNN	SVM / HOOM	E5 + *Naive Bayes* / MLP	mBERT / XLM-R / Reg. logística	BERTimbau + Grafos
Explicabilidade	Não	Intrínseca	Não aborda	LIME / SHAP (em exemplos)	SHAP / LIME / Grafos
Limitação	Caixa-Preta	Dados em inglês, privados	Baixa Performance	Sem medidas de qualidade das explicações	Custo Computacional

Fonte: Elaborado pelos autores

2.4.5 Limitações dos modelos existentes

2.4.5.1 O debate BERT versus SVM em domínios especializados

Wahba (2023), em sua tese de doutorado na Universidade de Western Ontario, propõe um contraponto relevante ao uso indiscriminado de modelos baseados em *transformers* para classificação de texto em domínios especializados. Com base em quatro anos de estudos empíricos com dados reais de uma empresa parceira, a autora demonstra que chamados de suporte de TI utilizam vocabulário técnico específico e de baixa ambiguidade. Nesse contexto, a principal vantagem do BERT, que é compreender palavras com múltiplos sentidos dependendo do contexto, tem impacto reduzido, tornando o SVM com TF-IDF competitivo em termos de desempenho.

Com base nessa observação, Wahba (2023) propõe o modelo HOOM (*Hybrid Online Offline Model* – Modelo Híbrido *Online* e *Offline*), que combina um classificador SVM linear treinado em modo *offline* com um classificador *Passive Aggressive Classifier*. Esse segundo classificador é um algoritmo de aprendizado incremental que atualiza o modelo a cada nova amostra, mantendo-se "passivo" quando a predição está correta e "agressivo" ao corrigir erros e treinado em modo *online* para detectar mudanças na distribuição dos dados ao longo do tempo. O modelo é descrito como mais eficiente, interpretável e reproduzível do que abordagens baseadas em *transformers*, que são modelos de linguagem baseados em redes neurais profundas e que serão detalhados na Seção 2.3.5.

Este argumento é diretamente relevante para o presente trabalho, pois os cenários A (SVM com TF-IDF) e B (BERTimbau) permitem medir, nos chamados analisados, o ganho do modelo mais complexo (Seção 3.2.1). Além do desempenho, investiga-se se a camada de explicabilidade traz um benefício de auditabilidade ao classificador.

2.4.5.2 A caixa-preta e a verificação das decisões em produção

Zemp (2021), em estudo desenvolvido na ZHAW com dados reais de um banco privado de médio porte com atuação internacional, treinou três tipos de modelos: CNN, BiLSTM, que é uma rede neural recorrente bidirecional capaz de processar sequências de texto nos dois sentidos, e RoBERTa, uma variante aprimorada do BERT com treinamento mais robusto. O estudo utilizou mais de um milhão de *tickets* históricos, atingindo, com a CNN, 92% de acurácia de validação e F1-macro de 0,75 em mais de 300 classes. O sistema foi parcialmente implantado em produção, e os agentes de suporte foram convidados a utilizá-lo em seu trabalho diário.

Na avaliação com a equipe de suporte, a previsão do serviço afetado foi considerada útil por unanimidade e mantida no sistema. Já a classificação entre requisições e incidentes, embora tenha atingido 93% de acurácia de validação, foi recusada: a equipe a considerou potencialmente perigosa, por poder levar a avaliações superficiais dos chamados, e entendeu que essa distinção deve ser aprendida pelos próprios analistas no trabalho.

Em um ambiente ITSM corporativo multilíngue com mais de 87 mil *tickets*, Razma e Jurkus (2026) aplicaram LIME e SHAP para gerar explicações das predições do modelo de regressão logística e, no caso do LIME, também do mBERT (multilingual BERT), uma versão do BERT treinada simultaneamente em mais de 100 idiomas, e afirmam que essas explicações contribuem para a interpretabilidade e a confiança em sistemas de automação de ITSM multilíngues.

Esses estudos ajudam a justificar a proposta deste trabalho: sistemas de classificação de chamados de TI em produção real funcionam, mas a ausência de mecanismos de explicabilidade limita sua auditabilidade. A proposta deste TCC busca lidar com essa limitação.

2.4.6 Lacunas identificadas no estado da arte

2.4.6.1 Ausência de XAI em sistemas de classificação de tickets em produção

O trabalho de Zemp (2021) demonstra que sistemas de classificação de chamados de TI com alta acurácia são tecnicamente viáveis e já foram implantados em produção. No entanto, a ausência de mecanismos de explicabilidade limita a auditabilidade desses sistemas. Nos modelos comparados por Zemp (2021), a predição não vem acompanhada de justificativa, o que dificulta a identificação de erros sistemáticos. Entre os trabalhos de classificação de chamados de TI identificados na pesquisa bibliográfica (Seção 2.1), nenhum combina explicações XAI com um grafo de co-ocorrência.

2.4.6.2 Carência de soluções robustas para PT-BR em *helpdesk*

Os trabalhos de Yokota (2025) e Pereira et al. (2023) evidenciam que o estado da arte em classificação de chamados de suporte em português brasileiro ainda é incipiente. Os resultados reportados (75% de precisão média com ML tradicional e F1-macro de até 61% com NB e MLP sobre *embeddings*) são inferiores a modelos baseados em *transformers* já alcançados em inglês, embora os dados e as métricas sejam diferentes. Não foi identificado nenhum trabalho nacional que aplique BERTimbau com XAI a chamados de suporte de TI.

2.4.6.3 Custo-benefício do BERT sem camada de explicabilidade

Wahba (2023) questiona o uso de BERT para classificação de texto em domínios especializados, argumentando que o custo computacional adicional não se justifica diante da pequena vantagem sobre o SVM em textos técnicos com baixa polissemia. Esse argumento permanece válido enquanto o único critério de avaliação for a acurácia. O presente trabalho examina esse argumento medindo o ganho do BERTimbau sobre o SVM e investigando se a camada de XAI e grafo de co-ocorrência contribui para a auditabilidade do processo de classificação.

2.4.7 Direcionamento para a proposta

Diante das lacunas identificadas: ausência de XAI em sistemas de classificação de chamados em produção, carência de soluções robustas para PT-BR em *helpdesk* de TI, e a questão do custo-benefício do BERT sem explicabilidade, o presente trabalho investiga um *pipeline* que combina BERTimbau, SHAP, LIME e grafo de co-ocorrência ponderado para classificação de chamados em PT-BR com explicabilidade visual. A descrição detalhada da arquitetura, do *dataset*, dos experimentos e da interface desenvolvida é apresentada no Capítulo 4.

3 Metodologia

Este capítulo descreve o plano da pesquisa: o tipo de estudo e a hipótese, as etapas da abordagem proposta, as ferramentas e os procedimentos experimentais com suas métricas. Os dados utilizados, os cenários executados e a implementação do sistema são detalhados no Capítulo 4.

3.1 Tipo de pesquisa e hipótese

Trata-se de uma pesquisa aplicada, de caráter empírico e comparativo: o estudo mede o desempenho de modelos de classificação e a qualidade das explicações produzidas sobre chamados reais de suporte de TI, comparando quatro cenários que acrescentam um componente por vez ao *pipeline* (Tabela 3). A hipótese de pesquisa, formulada na Seção 1.3.3, é examinada nos cenários C e D.

3.2 Procedimentos da abordagem proposta

A abordagem proposta segue as etapas abaixo, em ordem cronológica:

1. **Preparação dos dados**: anonimização dos chamados cedidos pelas empresas, filtragem dos chamados da Bayer em português e harmonização das categorias;
2. **Divisão dos dados**: divisão estratificada pela categoria e agrupada pelo texto do chamado em treino, validação e teste, com semente fixa e índices salvos, de modo que os cenários comparados usem os mesmos conjuntos;
3. **Cenário A**: treinamento do *baseline* SVM com TF-IDF;
4. **Cenário B**: *fine-tuning* do BERTimbau;
5. **Cenário C**: cálculo das explicações SHAP e LIME sobre uma amostra de chamados do conjunto de teste;
6. **Cenário D**: construção do grafo de co-ocorrência com os *scores* dessas explicações;
7. **Avaliação**: cálculo das métricas de classificação e das medidas de qualidade das explicações (Seção 3.4.1).

Os dados e a implementação de cada etapa são detalhados no Capítulo 4.

3.2.1 Cenários experimentais

O experimento foi estruturado em quatro cenários comparativos, conforme descrito na Tabela 3, com o objetivo de isolar a contribuição de cada componente do *pipeline*:

Tabela 3 – Cenários experimentais para avaliação do sistema

Cenário	**Modelo**	**XAI**	**Objetivo**
A	SVM + TF-IDF	Nenhum	Baseline clássico
B	BERTimbau	Nenhum	Ganho do transformer
C	BERTimbau	SHAP + LIME	Qualidade das explicações
D	BERTimbau	SHAP + LIME + grafo	Auditabilidade visual

Fonte: Elaborado pelos autores

A comparação entre os cenários A e B avalia o ganho de desempenho do BERTimbau sobre o *baseline* clássico, respondendo ao argumento de Wahba (2023) sobre o custo-benefício dos *transformers* em domínios especializados. Os cenários C e D examinam a hipótese de pesquisa (Seção 1.3.3), verificando se a camada de grafo de co-ocorrência contribui para a auditabilidade, nos termos definidos na Seção 2.2.6, e se o SHAP e o LIME diferem quanto à fidelidade e à estabilidade de suas explicações.

3.3 Ferramentas, tecnologias e ambientes

O projeto foi desenvolvido em Python. A Tabela 4 resume as bibliotecas utilizadas em cada etapa e o motivo da escolha.

O treinamento dos modelos usou Python 3.13.2, PyTorch 2.6.0 (CUDA 12.4), Transformers 5.17.0 e *scikit-learn* 1.9.1. A versão do SHAP utilizada foi a 0.54.1, e a do LIME, a 0.2.0.1.

O BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020) foi escolhido por ser um modelo BERT pré-treinado em português brasileiro. O *baseline* SVM com TF-IDF foi escolhido por ser a referência clássica em classificação de texto e permitir medir o ganho do modelo neural.

O *fine-tuning* do BERTimbau foi executado em uma placa de vídeo (GPU) NVIDIA GTX 1650.

Já a execução do SHAP e do LIME foi feita em CPU, em um MacBook Pro com chip Apple M4.

Tabela 4 – Bibliotecas utilizadas em cada etapa do projeto

Etapa	Biblioteca	Finalidade
Preparação dos dados	*pandas*, *openpyxl*, *pyarrow*, *spaCy*, *PyYAML*	Leitura das planilhas, limpeza, anonimização e armazenamento dos chamados
Classificador clássico (cenário A)	*scikit-learn*	TF-IDF e SVM linear, usados como *baseline*
Classificador neural (cenário B)	*PyTorch*, *Transformers*	*Fine-tuning* e inferência do BERTimbau
Explicabilidade (cenários C e D)	*SHAP*, *LIME*	Importância das palavras para a classe prevista
Métricas e análise	*NumPy*, *SciPy*, *scikit-learn*	Métricas de classificação, Jaccard e Spearman
Grafo e interface	*React*, *Vite*, *vis-network*, *Flask*	Exibição do grafo de co-ocorrência e comunicação da interface com o modelo

Fonte: Elaborado pelos autores

3.4 Procedimentos experimentais e métricas

3.4.1 Métricas de desempenho do classificador

O desempenho dos classificadores (cenários A e B) é medido no conjunto de teste, separado do treino e da validação, por meio das métricas abaixo, amplamente utilizadas em classificação de texto.

- **Acurácia (*Accuracy*)** – proporção de classificações corretas sobre o total de instâncias avaliadas.
- **Precisão (*Precision*)** – proporção de classificações corretas entre os casos previstos para uma determinada categoria, medindo a capacidade do modelo de evitar falsos positivos.
- **Revocação (*Recall*)** – capacidade do modelo de identificar corretamente os chamados pertencentes a cada categoria, medindo a capacidade de evitar falsos negativos.
- **F1-score** – média harmônica entre precisão e revocação, expressa pela fórmula $F1 = 2 \cdot \frac{\text{Precisão} \times \text{Revocação}}{\text{Precisão} + \text{Revocação}}$. Utiliza-se a variante **F1-macro**, que calcula o F1-score de cada categoria e tira a média, tratando todas as categorias com o mesmo peso, independentemente do tamanho. Isso é especialmente importante em conjuntos de dados desbalanceados.

- **Matriz de confusão** – representação tabular que detalha os tipos de erros cometidos pelo modelo, permitindo identificar padrões sistemáticos de confusão entre categorias semanticamente próximas.

3.4.2 Critérios de validação

O sucesso da solução é avaliado sob duas perspectivas complementares.

Desempenho da classificação. O critério definido no TCC1 é que o BERTimbau supere o *baseline* SVM com TF-IDF (cenário A) em pelo menos 5 pontos percentuais de F1-macro. A diferença é calculada no conjunto de teste, com intervalo de confiança de 95% obtido por *bootstrap* pareado (2.000 reamostragens, semente 42), em que os chamados com texto idêntico são reamostrados juntos, e é reportada como resultado do estudo, seja ela maior ou menor que o critério. Os mesmos cálculos são repetidos no conjunto de validação.

Qualidade das explicações. SHAP e LIME são comparados sobre uma amostra de 98 chamados do conjunto de teste, entre acertos e erros do modelo, sem textos repetidos. Cada campo (descrição curta e descrição completa) é explicado separadamente, com o outro fixo, e a perturbação é a remoção de palavras. As propriedades avaliadas são:

- **Fidelidade:** mede se as palavras apontadas como importantes realmente influenciam a decisão do modelo. Usam-se a *comprehensiveness*, a *sufficiency* (AOPC) e o Tau-LOO, comparados a uma linha de base aleatória. As medidas seguem a definição de Attanasio et al. (2023), mas foram implementadas pelos autores, pois a biblioteca *ferret* não pôde ser instalada no ambiente utilizado.

- **Estabilidade:** mede se a explicação se mantém ao repetir o método sobre o mesmo chamado. Repete-se cada explicação cinco vezes e calculam-se a similaridade de Jaccard das cinco palavras mais relevantes, a correlação de Spearman e a fração de repetições com a mesma palavra no topo, em chamados com ao menos oito palavras distintas.

- **Concordância entre SHAP e LIME:** mede se os dois métodos apontam as mesmas palavras, e no mesmo sentido, para o mesmo chamado. Calculam-se a correlação de Spearman entre os pesos das palavras, a similaridade de Jaccard das cinco palavras mais relevantes, a fração de chamados com a mesma palavra no topo e a fração de palavras com o mesmo sinal (a favor ou contra a classe prevista), cada medida comparada ao valor esperado ao acaso.

- **Coerência semântica:** análise qualitativa, nos estudos de caso, de se os termos destacados pelo SHAP e pelo LIME correspondem ao vocabulário técnico esperado para a categoria predita.

Grafo de co-ocorrência. A auditabilidade do grafo é avaliada pela sua rastreabilidade: cada nó e cada aresta pode ser levado de volta ao chamado original, à janela de palavras que o gerou e às notas do SHAP e do LIME.

4 Desenvolvimento

Neste capítulo, apresentam-se a modelagem e a arquitetura da solução, os detalhes de implementação, os experimentos realizados e o tratamento dos dados utilizados. O cronograma das atividades está no Apêndice A.

4.1 Modelagem e arquitetura

4.1.1 Visão geral da solução

A solução consiste em um *pipeline* que classifica chamados de suporte técnico e explica cada decisão, utilizando técnicas de Processamento de Linguagem Natural (PLN) e Inteligência Artificial Explicável (XAI). O *pipeline* é o meio para investigar a auditabilidade do processo decisório: por meio dele, compara-se a qualidade das explicações geradas por SHAP e LIME e avalia-se como o grafo de co-ocorrência permite rastrear quais termos do chamado influenciaram a classificação.

Para isso, o *pipeline* utiliza duas técnicas complementares de explicabilidade: o SHAP, que atribui a cada termo do chamado um valor numérico indicando o quanto ele contribuiu para a classificação, e o LIME, que gera explicações locais testando versões modificadas do texto. Os valores SHAP e LIME são utilizados como pesos dos nós em um grafo de co-ocorrência $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, produzindo uma visualização gráfica que combina a estrutura relacional do texto com a relevância explicativa de cada termo segundo ambos os métodos.

A interface é uma prova de conceito do *pipeline*, voltada a gestores e analistas de TI, e não foi avaliada junto a esse público neste trabalho.

4.1.2 Protótipos da interface

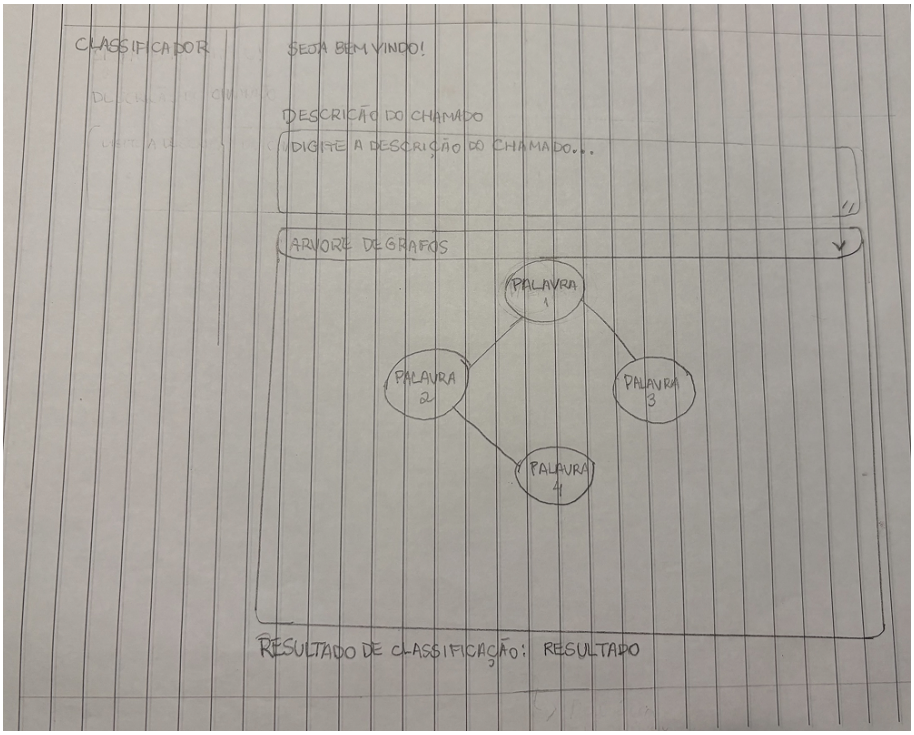
Durante a etapa inicial do projeto foram elaborados protótipos de baixa e média fidelidade com o objetivo de validar a organização visual da interface, o fluxo de navegação e a disposição das funcionalidades relacionadas à explicabilidade do modelo.

A etapa de prototipação de baixa fidelidade teve início com a elaboração individual de diferentes propostas de interface por cada integrante do grupo, permitindo que distintas ideias e abordagens fossem exploradas inicialmente. Posteriormente, as principais características de cada proposta foram analisadas e consolidadas em uma versão única do protótipo que passou a servir como base para a continuidade do projeto.

Os protótipos de baixa fidelidade foram utilizados para estruturar os principais componentes da aplicação, definindo a posição dos menus, área de inserção de chamados, painel

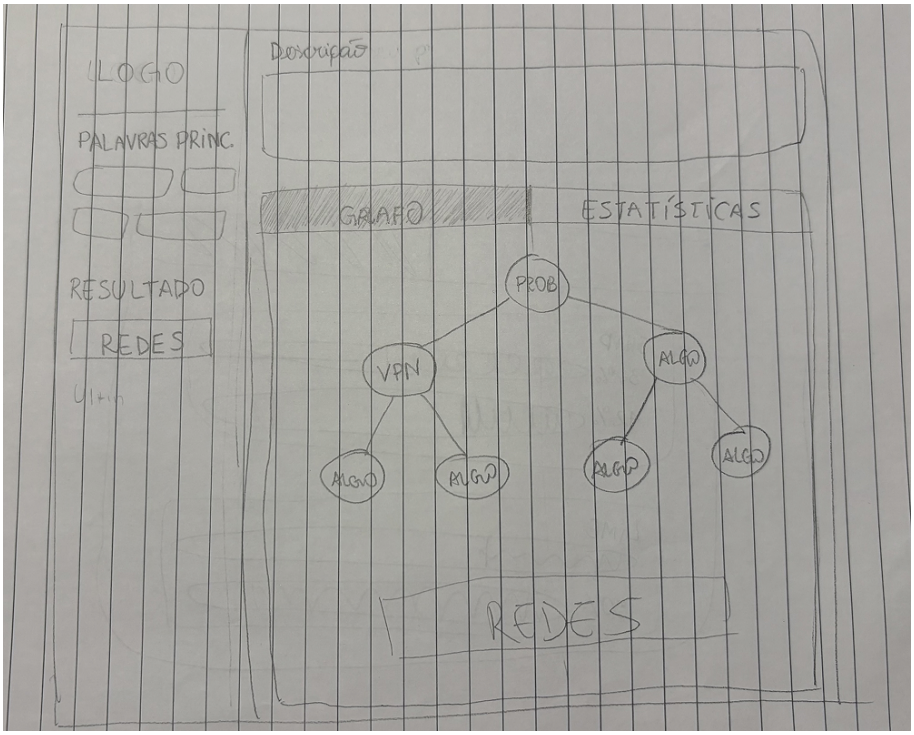
de resultados e espaço destinado às visualizações explicativas.

Figura 6 – Protótipo de baixa fidelidade da interface (proposta 1)



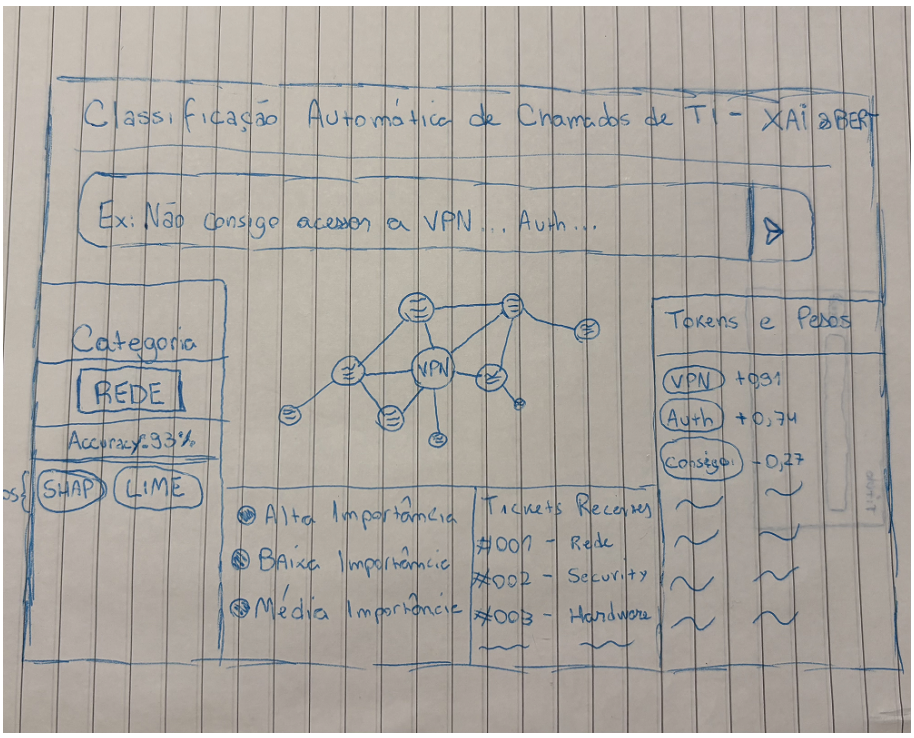
Fonte: Elaborado pelos autores

Figura 7 – Protótipo de baixa fidelidade da interface (proposta 2)



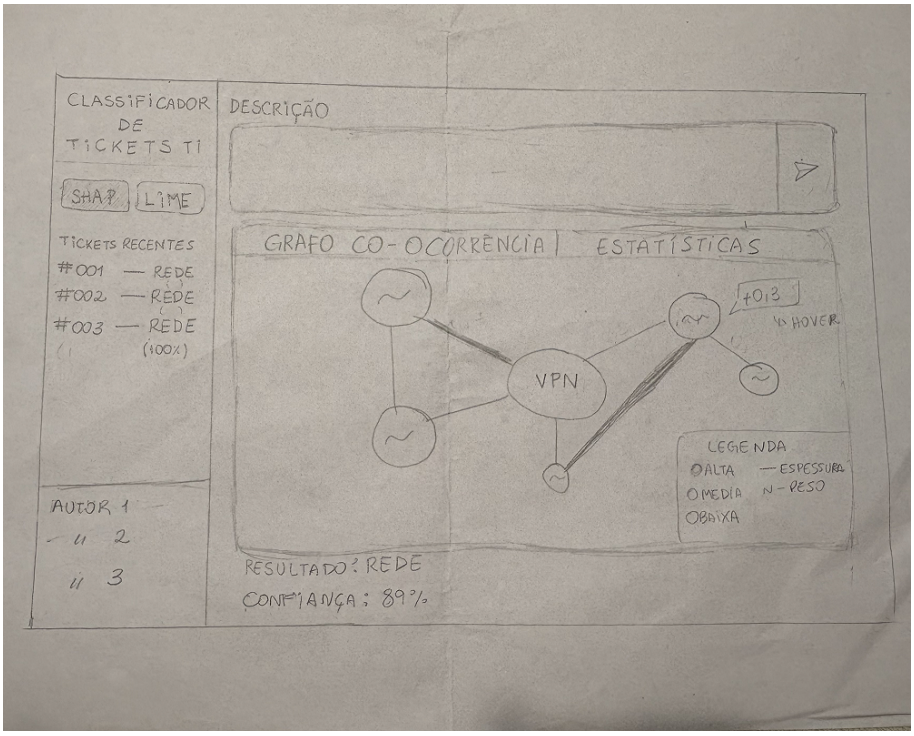
Fonte: Elaborado pelos autores

Figura 8 – Protótipo de baixa fidelidade da interface (proposta 3)



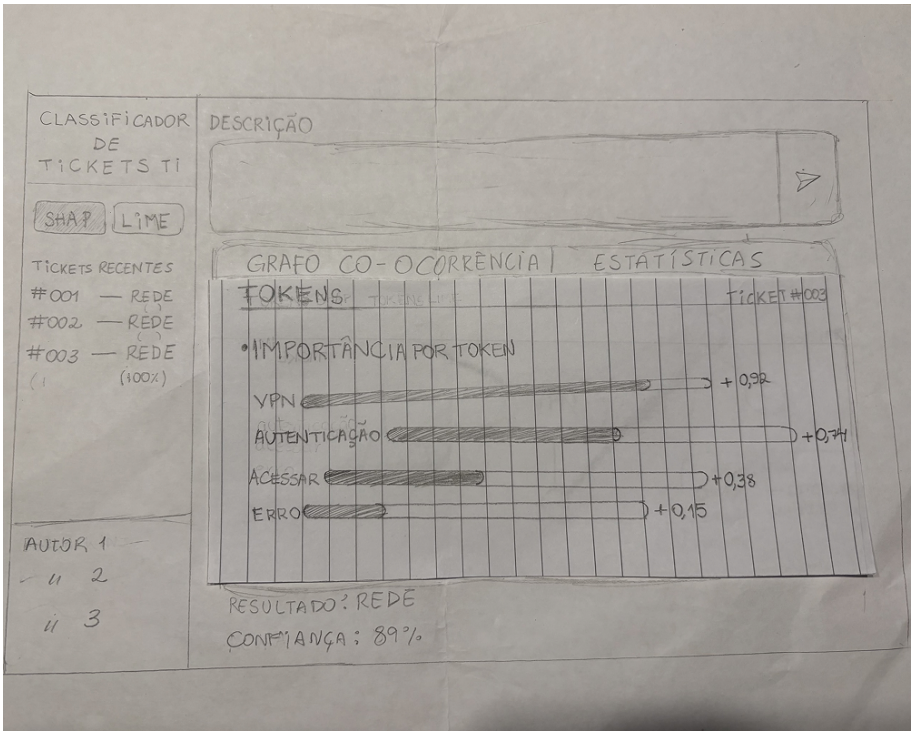
Fonte: Elaborado pelos autores

Figura 9 – Protótipo de baixa fidelidade da interface final - Tela do grafo



Fonte: Elaborado pelos autores

Figura 10 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas



Fonte: Elaborado pelos autores

Posteriormente, foram elaborados protótipos de média fidelidade com maior nível de detalhamento visual, com foco em validar como as explicações geradas pelo modelo seriam apresentadas de forma compreensível para os analistas de TI.

A área central do sistema é dedicada às explicações da classificação, organizada em duas abas principais, ambas voltadas para tornar o processo decisório do modelo auditável:

- **Grafo de Co-Ocorrência**: representação visual das relações entre os *tokens* do chamado, onde o tamanho de cada nó é proporcional à importância do termo (módulo do valor SHAP ou LIME, conforme o método escolhido em um seletor da interface), e a espessura das conexões indica a frequência de co-ocorrência entre os *tokens*;
- **Estatísticas**: painel que apresenta numericamente a importância individual de cada *token* calculada pelos métodos SHAP e LIME, permitindo ao analista comparar e validar as contribuições de cada termo na decisão do modelo.

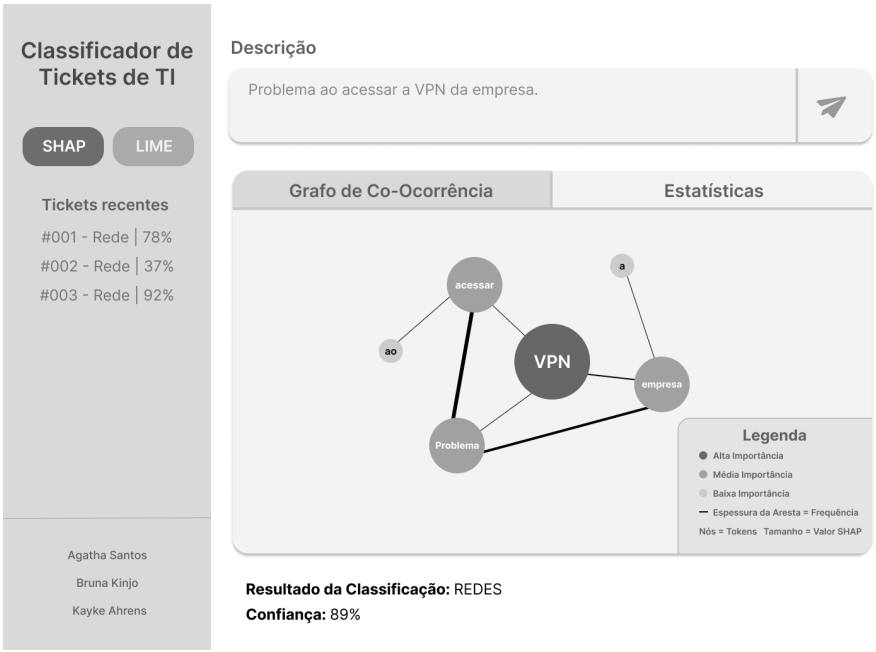
A escolha por disponibilizar tanto visualizações gráficas quanto estatísticas numéricas busca garantir que analistas com diferentes perfis consigam auditar as decisões do modelo, seja por meio de padrões visuais no grafo ou por comparações quantitativas entre os valores de importância dos *tokens*.

Na lateral esquerda do sistema encontra-se a seção “Tickets recentes”, responsável por apresentar classificações realizadas anteriormente juntamente com seus respectivos

níveis de confiança. Esse recurso permite consultar rapidamente chamados processados anteriormente e auxilia na análise das classificações realizadas.

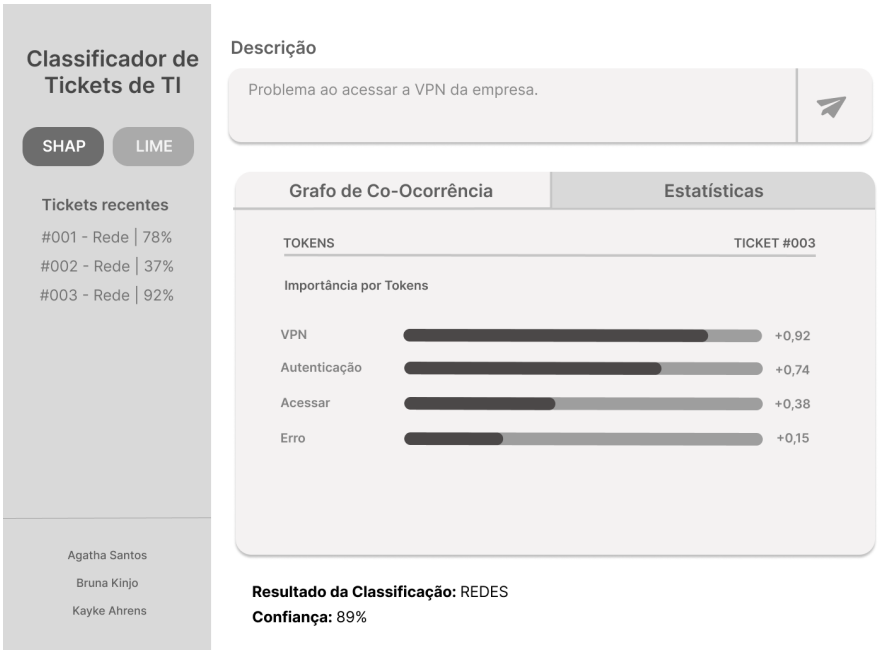
Os protótipos desenvolvidos tiveram como principal objetivo verificar se os recursos de explicabilidade seriam apresentados de forma intuitiva, clara e acessível aos analistas de TI.

Figura 11 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência



Fonte: Elaborado pelos autores

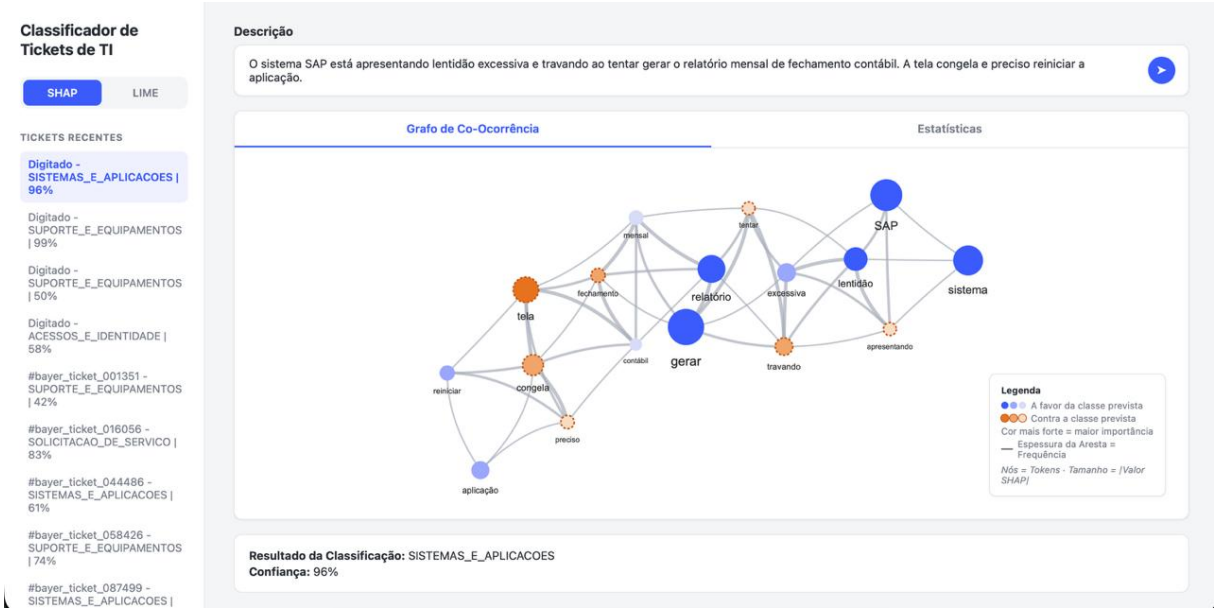
Figura 12 – Protótipo de média fidelidade - visualização estatística dos *tokens*



Fonte: Elaborado pelos autores

A Figura 13 mostra a interface final com um chamado inventado, digitado apenas para ilustração: a interface exibe a categoria prevista e a confiança, e o grafo de co-ocorrência, com nós em azul (a favor da classe prevista) e laranja (contra), o seletor entre SHAP e LIME e a lista de chamados recentes.

Figura 13 – Interface final: classificação de um chamado digitado e grafo de co-ocorrência



Fonte: Elaborado pelos autores

4.1.3 Arquitetura da solução

A arquitetura da solução é organizada em três camadas principais:

- Camada de apresentação (*front-end*);
- Camada de processamento;
- Camada de dados.

A camada de apresentação é responsável pela interação com o usuário final, disponibilizando os formulários de inserção de chamados, exibição das classificações, visualização dos grafos e acesso ao histórico de *tickets* processados.

A camada de processamento é responsável pela execução das etapas de pré-processamento textual, inferência do modelo BERTimbau, cálculo das explicações utilizando SHAP e LIME, além da construção dos grafos de co-ocorrência.

Já a camada de dados é responsável pelo armazenamento das informações relacionadas aos chamados classificados e aos resultados gerados pelo sistema.

O fluxo geral de funcionamento da aplicação segue as seguintes etapas:

1. **Entrada do modelo** — a descrição curta e a descrição completa do chamado, sem remoção de *stopwords*, são passadas como um par de textos ao tokenizador do BERTimbau, que as converte em *tokens* com seus respectivos IDs e máscaras de atenção. A remoção de *stopwords* é usada apenas na construção do grafo.

2. **Classificação com BERTimbau** — o modelo BERTimbau com *fine-tuning* processa a sequência de *tokens* e produz, a partir da representação do *token* [CLS], as probabilidades de pertencimento a cada categoria. A categoria com maior probabilidade é selecionada como predição, e o índice de confiança corresponde a essa probabilidade máxima.

3. **Cálculo de explicabilidade** — sobre o mesmo texto de entrada, são calculados os valores SHAP e as importâncias LIME. Cada *token* recebe um valor numérico que representa sua contribuição para a predição: valores positivos indicam contribuição para a classe predita, e valores negativos indicam contribuição contrária.

4. **Construção do grafo de co-ocorrência** — os *tokens* do chamado são representados como nós de um grafo $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$. As arestas conectam pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho $w = 4$, com pesos proporcionais à frequência de co-ocorrência. Os pesos dos **nós** são determinados pelos valores SHAP e LIME calculados na etapa anterior — *tokens* com maior importância, segundo o método selecionado (SHAP ou LIME), geram nós visualmente maiores na interface. O grafo é serializado em JSON, com os campos `tokens` (nós) e `arestas`, e enviado ao *frontend*.

5. **Apresentação ao analista** — a interface recebe o JSON, renderiza o grafo e exibe a categoria predita, o índice de confiança, os *tokens* destacados por cor (azul, a favor da classe prevista; laranja, contra) e os valores SHAP e LIME comparativos.

4.2 Detalhes de implementação

4.2.1 Tecnologias e ferramentas

As tecnologias e bibliotecas listadas a seguir representam as ferramentas utilizadas para o desenvolvimento do projeto. O detalhamento por etapa está na Tabela 4.

Interface e visualização:

- *React* e *Vite*;
- *vis-network*, para a exibição do grafo;
- CSS.

Processamento e Inteligência Artificial:

- Python;
- *PyTorch* e *Transformers*, para o BERTimbau;
- *scikit-learn*, para o TF-IDF e o SVM;
- *SHAP* e *LIME*;
- *Flask*, para a comunicação entre a interface e o modelo.

Ambiente de desenvolvimento:

- Git e GitHub;
- Máquinas locais dos integrantes do grupo.

4.2.2 Funcionalidades principais

Entre as principais funcionalidades implementadas no sistema estão:

- **Funcionalidades de explicabilidade:**
  - Construção de grafos de co-ocorrência;
  - Visualização das explicações do modelo;
  - Seleção do método de explicação (SHAP ou LIME) exibido no grafo.
- **Funcionalidades de classificação e suporte:**
  - Entrada do chamado em um único campo de descrição;
  - Classificação automática de chamados de TI;
  - Exibição da categoria prevista;
  - Apresentação do nível de confiança da classificação;
  - Histórico simplificado de *tickets* classificados.

4.2.3 Metodologia de implementação

A implementação do projeto seguiu uma abordagem iterativa, baseada em prototipação e validações incrementais.

Inicialmente foram realizados o levantamento bibliográfico, a definição dos requisitos e a elaboração dos protótipos da interface. Em seguida, foram preparados os dados, treinados os classificadores dos cenários A e B e implementados os módulos de explicabilidade (SHAP e LIME) e de construção do grafo de co-ocorrência. Por fim, esses módulos foram integrados à interface, que passou a gerar a explicação de um chamado digitado pelo *pipeline* completo.

4.3 Experimentos e cenários de teste

Os experimentos seguiram os quatro cenários da Tabela 3, todos sobre os mesmos conjuntos de treino, validação e teste descritos na Seção 4.4.2.

4.3.1 Cenários A e B: classificação

No cenário A, o *baseline* SVM com TF-IDF foi treinado no conjunto de treino e avaliado no conjunto de teste, com a descrição curta e a descrição completa concatenadas como entrada. O TF-IDF usou unigramas e bigramas, frequência mínima de documento igual a 2 e escala sublinear; o classificador foi um *LinearSVC* com $C = 1$, sem pesos de classe e semente 42. Assim como no BERTimbau, não foram removidas *stopwords*. No cenário B, o BERTimbau foi ajustado com a configuração da Seção 4.4.2. Os dois classificadores foram comparados pelas métricas da Seção 3.4.1, com destaque para o F1-macro.

4.3.2 Cenário C: explicações SHAP e LIME

As explicações foram geradas em 01/11/2026, para o modelo final, sobre uma amostra de 98 chamados do conjunto de teste: 8 acertos de cada uma das 10 classes que tiveram acertos (80 chamados) e 18 erros do modelo. A classe OUTROS não teve acertos, e por isso a amostra não tem 11×8 acertos. Foi necessário usar uma amostra porque SHAP e LIME explicam uma predição por vez, e explicar os 5.293 chamados do teste levaria cerca de 24 horas.

O LIME usou 500 amostras por explicação e semente 42. O SHAP (versão 0.54.1) foi calculado de forma exata nos campos de até 10 palavras (87 campos); acima disso, foi estimado por amostragem, com orçamento de 500 consultas ao modelo (79 campos). A fidelidade e a estabilidade seguiram o protocolo da Seção 3.4.2; para a estabilidade, 12 chamados foram explicados 5 vezes cada. A execução completa levou cerca de 39 minutos, em CPU, em um computador Apple.

4.3.3 Cenário D: grafo de co-ocorrência

Para cada chamado explicado, construiu-se um grafo de co-ocorrência com janela deslizante de tamanho $w = 4$. As *stopwords* foram removidas apenas nessa etapa, e não da entrada do modelo. O peso de cada aresta é o número de co-ocorrências do par de palavras na janela, e o peso de cada nó é o módulo da soma dos pesos que o SHAP ou o LIME atribuiu à palavra. O grafo foi avaliado por rastreabilidade, em estudos de caso (Seção 3.4.2).

4.4 Coleta e tratamento de dados

4.4.1 Origem e autorização de uso dos dados

Os dados utilizados neste trabalho são chamados reais de suporte técnico cedidos por duas empresas do setor farmacêutico: Aché Laboratórios Farmacêuticos e Bayer S.A. Os autores solicitaram o acesso por e-mail, explicando o objetivo acadêmico do trabalho, quais informações seriam necessárias e como os registros seriam tratados.

Não foi utilizado um conjunto de dados público. Os chamados são dados privados das duas empresas, cedidos com finalidade acadêmica e anonimizados, e por isso os textos originais não são divulgados neste trabalho.

As autorizações de uso foram concedidas pelas duas empresas por e-mail. Os autores se comprometeram a usar os chamados apenas neste trabalho e a anonimizá-los antes de qualquer etapa de processamento. Essas mensagens são o registro da autorização concedida e da finalidade acadêmica do uso, e estão reproduzidas nos Anexos A e B.

A anonimização, além de cumprir esse compromisso, alinha o trabalho ao tratamento que a Lei Geral de Proteção de Dados Pessoais (LGPD, Lei n.º 13.709/2018) dá aos dados anonimizados, que não são considerados dados pessoais para os fins da lei, salvo quando a anonimização puder ser revertida com esforços razoáveis (BRASIL, 2018). A etapa descrita na Seção 4.4.2 deixa de ser, portanto, apenas uma escolha metodológica.

4.4.2 *Dataset* e *fine-tuning*

O modelo BERTimbau, em sua versão *base*, foi ajustado com chamados reais da Bayer Brasil e da Aché, obtidos mediante autorização concedida por e-mail e anonimização dos registros. O arquivo da Bayer tinha 191.350 chamados, abertos de janeiro de 2021 a agosto de 2026; após a remoção de duplicatas pelo número do chamado, o descarte de chamados sem categoria ou de categorias com menos de 100 chamados e a anonimização, restaram 189.909, dos quais 46.584 estavam em português e foram mantidos por um filtro de idioma. O arquivo da Aché tinha 9.317 chamados, todos de abril de 2026; após o descarte das categorias com menos de 100 chamados (restaram 8.758), dos chamados sem descrição (715) e de 326 cópias de uma mensagem automática de abertura por e-mail, todas da categoria de cibersegurança, restaram 7.717. As categorias das duas empresas foram harmonizadas em uma taxonomia única, e o conjunto combinado, com 54.301 chamados, passou ainda pelo descarte da classe com menos de 100 chamados (Recursos Humanos, 39 chamados) e pela remoção de 983 chamados, em 75 grupos de textos idênticos com categorias divergentes. Os resultados foram 53.279 chamados (Aché: 6.949; Bayer: 46.330), em 11 classes, entre elas a classe OUTROS.

As 11 classes da taxonomia harmonizada estão na Tabela 5.

A base da Bayer possui, além da descrição completa, uma descrição curta, que a base

Tabela 5 – Categorias de classificação dos chamados

Categoria	Tipo de chamado (exemplo)	Chamados no teste
SISTEMAS_E_APLICACOES	Falhas e dúvidas em sistemas corporativos e aplicações	2.458
ACESSOS_E_IDENTIDADE	Criação, bloqueio e revogação de acessos e contas (ex.: desligamento de colaborador)	610
SUPORTE_E_EQUIPAMENTOS	Suporte a computadores e equipamentos (ex.: *checklist* preventivo de salas de reunião)	589
CONSULTA_E_ORIENTACAO	Dúvidas e pedidos de orientação	555
REDES_E_COMUNICACOES	Rede, conectividade e comunicação	435
SOFTWARE	Instalação e problemas de *software*	297
SOLICITACAO_DE_SERVICO	Pedidos de serviço (ex.: limpeza de um *rack*)	260
SERVIDORES	Chamados sobre servidores	40
CIBERSEGURANCA	Segurança da informação	21
AQUISICAO_TI	Compra e licenciamento (ex.: transferência de licença do *Office*)	16
OUTROS	Demais chamados (ex.: instalação de impressoras)	12

Fonte: Elaborado pelos autores

da Aché não tem; nesse caso, o campo entra vazio. Ainda há chamados com texto repetido nas bases, sobretudo na Aché, o que motivou o agrupamento descrito a seguir.

O conjunto combinado foi dividido em 42.692 chamados de treino, 5.294 de validação e 5.293 de teste (proporção aproximada de 80/10/10). A divisão usou *StratifiedGroupKFold* (8 partes para treino, 1 para validação e 1 para teste), estratificada pela categoria e agrupada pelo texto do chamado: chamados com texto idêntico ficam sempre na mesma parte.

O processo de *fine-tuning* seguiu uma abordagem supervisionada de classificação textual. A entrada do modelo é o par formado pela descrição curta e pela descrição completa, limitado a 256 *tokens*; cerca de 10,2% dos pares ultrapassam esse limite. O treino usou 3 épocas, *batch* de 16, taxa de aprendizado de 2×10^{-5} e semente 42. Para reduzir o risco de *overfitting*, fenômeno que ocorre quando o modelo aprende os dados de treino com precisão excessiva e perde a capacidade de generalizar para novos exemplos, o desempenho é acompanhado no conjunto de validação durante o treino. Não foram aplicadas técnicas de balanceamento de classes nem pesos por classe; o desbalanceamento é tratado na avaliação, com o uso do F1-macro.

5 Resultados e Análise

5.1 Apresentação dos resultados

5.1.1 Desempenho da classificação (cenários A e B)

A Tabela 6 apresenta o desempenho dos dois classificadores no conjunto de teste, com 5.293 chamados em 4.824 grupos de texto. O BERTimbau obteve F1-macro de 0,628 e o *baseline* SVM com TF-IDF, de 0,591. A acurácia dos dois é próxima (0,724 e 0,719); a diferença aparece no F1-macro, que dá o mesmo peso a todas as classes, e vem da revocação (0,611 contra 0,550), pois a precisão é parecida (0,655 contra 0,662).

Tabela 6 – Desempenho dos classificadores no conjunto de teste

Modelo	Acurácia	Precisão (macro)	Revocação (macro)	F1-macro
Cenário A: SVM + TF-IDF	0,719	0,662	0,550	0,591
Cenário B: BERTimbau	0,724	0,655	0,611	0,628

Fonte: Elaborado pelos autores

A diferença de F1-macro a favor do BERTimbau foi de 3,7 pontos percentuais, com intervalo de confiança de 95% de 1,0 a 6,7 (*bootstrap* pareado por grupo de texto, 2.000 reamostragens). A diferença é positiva com alta confiança (probabilidade de 99,8% nas reamostragens), mas a estimativa pontual fica abaixo do critério de 5 pontos definido no TCC1 (Seção 3.4.2). Como o intervalo inclui os 5 pontos, os dados também não permitem descartar que a diferença verdadeira os alcance: a probabilidade de a diferença ser de pelo menos 5 pontos é de cerca de 21%.

A Tabela 7 mostra o F1 por classe. O ganho do BERTimbau se concentra em quatro classes: SOFTWARE e SOLICITACAO_DE_SERVICO, com diferenças de 8,5 e 8,0 pontos e intervalos que não incluem o zero, e SERVIDORES e AQUISICAO_TI, classes pequenas (40 e 16 chamados) com intervalos largos. Juntas, essas quatro classes respondem por 3,55 dos 3,68 pontos de diferença no F1-macro. Na maior classe, SISTEMAS_E_APLICACOES (46% do teste), os dois modelos praticamente empatam (0,811 e 0,809). Nenhum dos dois acertou chamados da classe OUTROS, que tem apenas 12 chamados no teste.

A Tabela 8 separa o teste por empresa. O F1-macro de cada empresa é calculado sobre as classes presentes em seus chamados (7 na Aché e 9 na Bayer). Os dois modelos têm desempenho muito maior na Aché, e a diferença a favor do BERTimbau é de 1,3 ponto na Aché, com intervalo que inclui o zero, e de 4,0 pontos na Bayer. Como a Bayer reúne 86% dos chamados de teste, o resultado geral reflete sobretudo essa empresa. Os 716 chamados da Aché formam apenas 337 grupos de texto, e um desses textos, um roteiro de desligamento

Tabela 7 – F1 por classe no conjunto de teste e diferença entre BERTimbau e SVM

Classe	Chamados	F1 BERTimbau	F1 SVM	Dif. (p.p.)	IC 95% (p.p.)
SISTEMAS_E_APLICACOES	2.458	0,811	0,809	+0,3	−0,8 a 1,3
ACESSOS_E_IDENTIDADE	610	0,802	0,775	+2,7	0,5 a 8,8
SUPORTE_E_EQUIPAMENTOS	589	0,722	0,731	−0,8	−3,8 a 1,8
CONSULTA_E_ORIENTACAO	555	0,429	0,429	0,0	−4,3 a 3,9
REDES_E_COMUNICACOES	435	0,726	0,709	+1,7	−1,3 a 5,1
SOFTWARE	297	0,568	0,483	+8,5	3,2 a 13,7
SOLICITACAO_DE_SERVICO	260	0,440	0,360	+8,0	1,8 a 14,1
SERVIDORES	40	0,667	0,613	+5,4	−4,9 a 17,3
CIBERSEGURANCA	21	0,900	0,923	−2,3	−8,8 a 0,0
AQUISICAO_TI	16	0,839	0,667	+17,2	−7,0 a 44,8
OUTROS	12	0,000	0,000	0,0	0,0 a 0,0

Fonte: Elaborado pelos autores. IC: intervalo de confiança por *bootstrap* pareado; p.p.: pontos percentuais.

de colaborador, aparece em 298 chamados da classe ACESSOS_E_IDENTIDADE, todos classificados corretamente pelos dois modelos. Sem esse grupo, restam 418 chamados da Aché, ainda nas mesmas 7 classes, e o F1-macro da empresa fica em 0,901 (BERTimbau) e 0,889 (SVM). A diferença continua de 1,3 ponto, o que mostra que esse grupo repetido não explica o resultado da Aché nem a diferença entre os modelos.

Tabela 8 – Desempenho no conjunto de teste, por empresa de origem

Subconjunto	Chamados	Grupos	F1 BERTimbau	F1 SVM	Dif. (p.p.)	IC de 95% (p.p.)
Aché	716	337	0,911	0,897	+1,3	−3,1 a 5,9
Bayer	4.577	4.487	0,515	0,474	+4,0	1,6 a 6,5
Total	5.293	4.824	0,628	0,591	+3,7	1,0 a 6,7

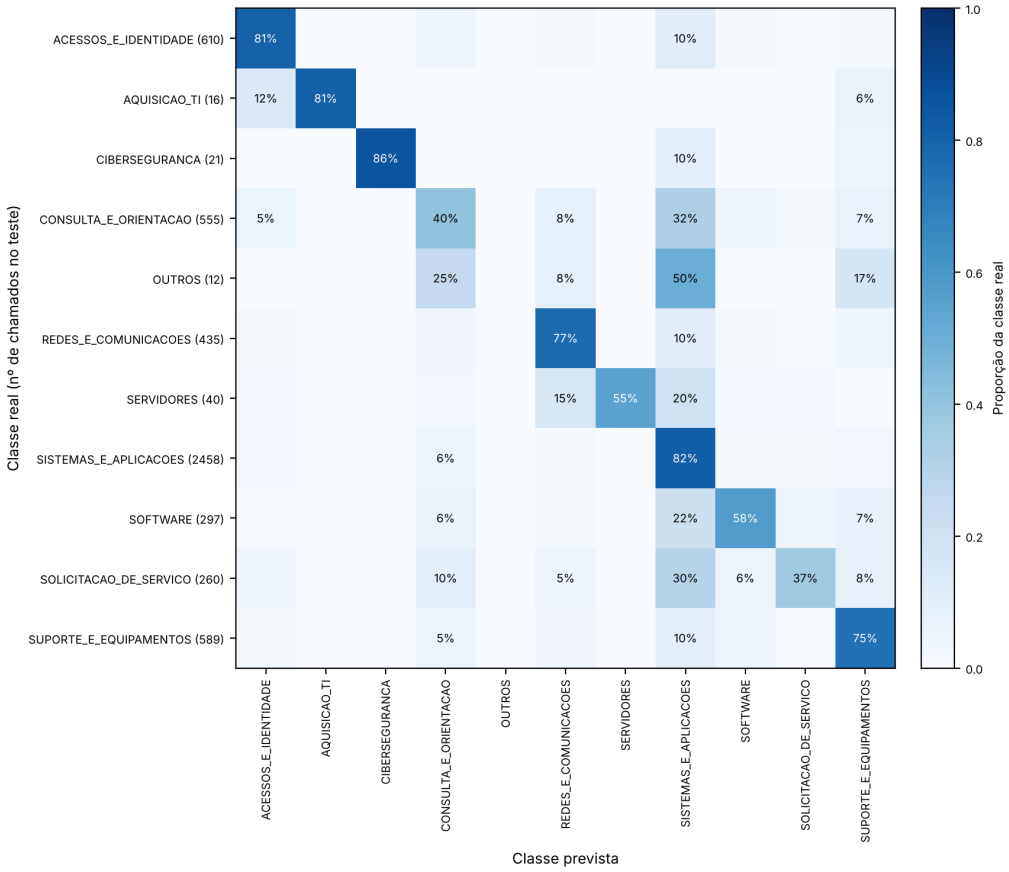
Fonte: Elaborado pelos autores. Grupos: conjuntos de chamados com texto idêntico.

No conjunto de validação, a diferença segue a mesma direção. Após retirar um bloco de 491 chamados de SUPORTE_E_EQUIPAMENTOS com o mesmo texto (um *checklist* preventivo de salas de reunião, que o SVM classificou como REDES_E_COMUNICACOES e o BERTimbau como SUPORTE_E_EQUIPAMENTOS), restam 4.803 chamados, nos quais o F1-macro foi de 0,591 para o BERTimbau e 0,555 para o SVM, uma diferença de 3,5 pontos (intervalo de 95% de 1,3 a 6,1). Na validação inteira, de 5.294 chamados, a diferença seria de 11,9 pontos, inflada por esse único texto.

A Figura 14 mostra a matriz de confusão do BERTimbau no teste. Os erros mais frequentes ocorrem entre CONSULTA_E_ORIENTACAO e SISTEMAS_E_APLICACOES (180 chamados previstos como a segunda e 146 no sentido inverso), seguidos de SOLICITACAO_DE_SERVICO previstos como SISTEMAS_E_APLICACOES (78) e de SISTEMAS_E_APLICACOES previstos como SUPORTE_E_EQUIPAMENTOS (76). As classes de menor F1 são CONSULTA_E_ORIENTACAO (0,429), SOLICITACAO_DE_SERVICO (0,440) e OUTROS (0,000). Uma possível explicação, que este

trabalho não testou, é que essas classes descrevem o tipo de pedido, e não o sistema ou equipamento envolvido, e por isso se sobrepõem às demais.

Figura 14 – Matriz de confusão do BERTimbau no conjunto de teste, normalizada por classe real



Fonte: Elaborado pelos autores

5.1.2 Explicabilidade: SHAP e LIME (cenário C)

As explicações do SHAP e do LIME foram geradas para os 98 chamados da amostra (Seção 3.4.2). Em CPU, o LIME levou 964 segundos (cerca de 16 minutos) e o SHAP, 616 segundos (cerca de 10 minutos). No SHAP, a soma dos pesos das palavras com a probabilidade-base reproduziu a probabilidade do chamado em todas as explicações, com erro máximo da ordem de 10^{-15}. No LIME, o R^2 do modelo linear local teve mediana de 0,68, com mínimo de 0,11.

5.1.2.1 Fidelidade

A Tabela 9 traz a fidelidade dos dois métodos nos 98 chamados. Na *comprehensiveness*, mede-se a queda da probabilidade da classe prevista ao retirar as palavras apontadas pelo método (quanto maior, melhor); na *sufficiency*, a queda ao manter apenas essas palavras

(quanto menor, melhor); o Tau-LOO é a correlação de Kendall entre o peso de cada palavra e a queda de probabilidade ao retirá-la sozinha (quanto maior, melhor). As duas primeiras são comparadas à queda obtida com palavras sorteadas.

Tabela 9 – Fidelidade das explicações do SHAP e do LIME nos 98 chamados da amostra

Medida	Método	Explicação	Aleatório	Acima do acaso
Comprehensiveness (maior é melhor)	LIME	0,542	0,192	94 de 98
	SHAP	0,526	0,182	96 de 98
Sufficiency (menor é melhor)	LIME	0,019	0,309	96 de 98
	SHAP	0,010	0,317	93 de 98
Tau-LOO (maior é melhor)	LIME	0,412	–	95 de 98
	SHAP	0,353	–	95 de 98

Fonte: Elaborado pelos autores. Valores médios por chamado. Cinco sorteios aleatórios por chamado, semente 42. "Acima do acaso": chamados em que a explicação supera o sorteio. No Tau-LOO não há linha de base aleatória, e a última coluna indica os chamados com valor positivo.

Os dois métodos ficam bem acima do acaso nas três medidas. Retirar as palavras apontadas reduz a probabilidade da classe prevista em 0,54 (LIME) e 0,53 (SHAP), contra 0,19 e 0,18 ao retirar palavras sorteadas, e manter só as palavras apontadas deixa a probabilidade quase inalterada (queda de 0,02 e 0,01), contra 0,31 e 0,32 com palavras sorteadas. As diferenças entre os métodos são pequenas: o LIME tem valores um pouco maiores na *comprehensiveness* e no Tau-LOO (0,54 contra 0,53 e 0,41 contra 0,35), e o SHAP, uma queda menor na *sufficiency* (0,010 contra 0,019). Para comparar os métodos nos mesmos chamados, calculou-se a diferença SHAP menos LIME em cada chamado, com intervalo de confiança de 95% por *bootstrap* sobre os chamados (5.000 reamostragens, semente 42). Na *comprehensiveness*, a diferença média foi de $-0{,}015$ (intervalo de $-0{,}037$ a 0,006), e na *sufficiency*, de $-0{,}008$ (intervalo de $-0{,}033$ a 0,012); os dois intervalos incluem o zero, e os dados não distinguem os métodos nessas medidas. No Tau-LOO, a diferença foi de $-0{,}059$ (intervalo de $-0{,}090$ a $-0{,}027$), ou seja, o LIME teve correlação maior com o efeito de retirar cada palavra.

5.1.2.2 Estabilidade

A estabilidade foi medida em 12 chamados com ao menos oito palavras distintas, explicados 5 vezes cada, com sementes diferentes (Tabela 10). Em 3 desses chamados, todos os campos têm até 10 palavras e o SHAP foi calculado de forma exata, sem sorteio, de modo que a explicação é idêntica nas cinco repetições (estabilidade 1 por construção). Nos outros 9, ao menos um campo foi estimado por amostragem, com orçamento de 500 consultas, e o SHAP variou de uma repetição para outra.

Tabela 10 – Estabilidade das explicações do SHAP e do LIME em 12 chamados, com 5 repetições

Método	Chamados	Jaccard (acaso)	Spearman	Topo igual (acaso)
LIME	12	0,659 (0,213)	0,774	0,950 (0,062)
SHAP, todos	12	0,638 (0,213)	0,736	0,758 (0,062)
SHAP, só amostragem	9	0,517 (0,135)	0,648	0,678 (0,043)

Fonte: Elaborado pelos autores. Valores médios por chamado. Jaccard: das 5 palavras mais relevantes entre as repetições. "Topo igual": fração das repetições com a mesma palavra no topo. Entre parênteses, o valor esperado ao acaso. "Só amostragem" exclui os 3 chamados em que o SHAP foi exato.

Os dois métodos ficam acima do acaso nas três medidas, mas nenhum é perfeitamente estável. O LIME manteve a mesma palavra no topo em 95% das repetições e teve Jaccard de 0,659. O SHAP teve Jaccard de 0,638 e a mesma palavra no topo em 76% das repetições; considerando só os 9 chamados em que houve amostragem, os valores caem para 0,517 e 68%. Nos 9 chamados em que houve amostragem, a comparação é direta, com os mesmos chamados nos dois métodos: o LIME teve Jaccard de 0,546 contra 0,517 do SHAP, Spearman de 0,706 contra 0,648 e a mesma palavra no topo em 93% das repetições contra 68%. Nessa amostra, portanto, o SHAP estimado por amostragem, com 500 consultas, não foi mais estável que o LIME, e só o cálculo exato dá estabilidade total ao SHAP. Como são apenas 12 chamados e não houve teste estatístico, o resultado é indicativo.

5.1.2.3 Concordância entre SHAP e LIME

A concordância compara, para cada um dos 98 chamados, as palavras que o SHAP e o LIME apontaram como importantes para a classe prevista. Como os pesos estão em escalas diferentes (probabilidade no SHAP, coeficiente do modelo linear local no LIME), a comparação usa a ordem e o sinal das palavras, e não o valor dos pesos. A Tabela 11 mostra o resultado, ao lado do valor esperado ao acaso.

Tabela 11 – Concordância entre SHAP e LIME nos 98 chamados da amostra

Medida	Resultado	Ao acaso	Chamados acima do acaso
Correlação de Spearman entre os pesos (média)	0,704	0,000	98 de 98
Jaccard das 5 palavras mais relevantes (média)	0,575	0,226	91 de 96
Mesma palavra no topo (fração de chamados)	0,714 (70 de 98)	0,068	–
Palavras com o mesmo sinal (média)	0,792	0,500	97 de 98

Fonte: Elaborado pelos autores. O Jaccard considera apenas os 96 chamados com mais de 5 palavras.

Nas quatro medidas, os dois métodos concordam bem acima do acaso, mas longe da concordância total. Em 70 dos 98 chamados (71%) os dois colocam a mesma palavra no topo, contra 7% esperados ao acaso, e a mediana da correlação de Spearman é 0,757. A concordância não indica qual dos métodos está certo, apenas que apontam as mesmas palavras; a fidelidade de cada um é tratada à parte.

A Tabela 12 separa os chamados por forma de cálculo do SHAP e por acerto do modelo. A concordância é maior nos 27 chamados em que o SHAP foi calculado de forma exata (Spearman de 0,864) do que nos 71 em que houve amostragem (0,643). Os chamados do primeiro grupo são os mais curtos, então os dados não separam o efeito do método de cálculo do efeito do tamanho do texto. Em todos os chamados, a correlação entre o número de palavras e a concordância de Spearman é de $-0{,}67$ ($p < 0{,}001$): quanto mais palavras, menos os dois métodos concordam. Já entre acertos e erros do modelo, a concordância é praticamente a mesma (Spearman de 0,704 nos dois grupos), com amostra de erros pequena (18 chamados).

Tabela 12 – Concordância entre SHAP e LIME por subconjunto de chamados

Subconjunto	Chamados	Spearman	Jaccard	Mesmo topo	Mesmo sinal
Todos	98	0,704	0,575	0,714	0,792
SHAP exato	27	0,864	0,762	0,741	0,899
SHAP por amostragem	71	0,643	0,509	0,704	0,752
Acertos do modelo	80	0,704	0,583	0,700	0,788
Erros do modelo	18	0,704	0,535	0,778	0,815

Fonte: Elaborado pelos autores. Valores médios por chamado; “Mesmo topo” é a fração de chamados com a mesma palavra no topo. O Jaccard considera apenas chamados com mais de 5 palavras (25 no SHAP exato, 71 na amostragem, 79 nos acertos e 17 nos erros).

5.1.3 Grafo de co-ocorrência (cenário D)

O grafo foi avaliado por rastreabilidade em três chamados da amostra de 98, escolhidos para cobrir situações diferentes: um acerto com vocabulário típico da classe (caso 1), um erro do modelo (caso 2) e um acerto em que os dois métodos divergem nas palavras secundárias (caso 3). A Tabela 13 resume os três, e as Figuras 15, 16 e 17 mostram os grafos na interface.

Nos três casos, a rastreabilidade se cumpriu. Cada nó corresponde a uma palavra que aparece no texto do chamado, com o campo em que ocorre e os pesos do SHAP e do LIME; cada aresta liga dois nós existentes e tem como peso o número de vezes que o par co-ocorre na janela deslizante de 4 palavras, valor que pode ser recalculado a partir do texto.

No caso 1, o chamado pede a transferência de uma licença do pacote *Office* para outra pessoa. As duas palavras de maior peso a favor da classe são as mesmas nos dois métodos, *licença* (SHAP de 0,476; LIME de 0,447) e *office* (0,151 e 0,138), e são o vocabulário

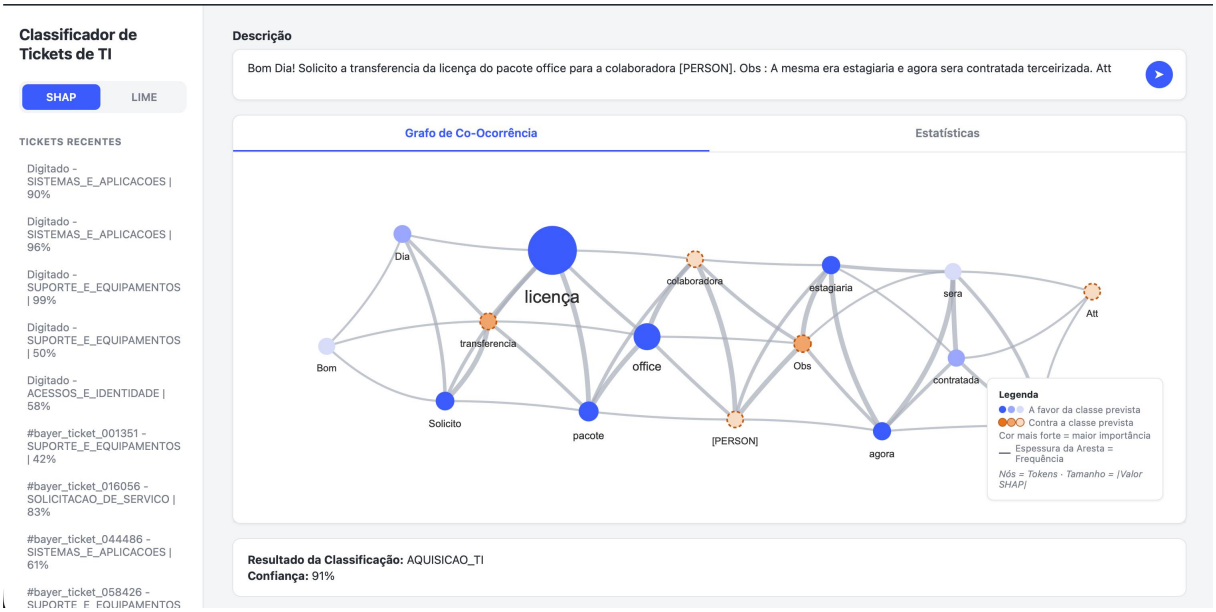
Tabela 13 – Chamados usados nos estudos de caso do grafo

Caso	Chamado	Classe real → prevista	Conf.	Nós/arestas
1	ache_ticket_001118	AQUISICAO_TI → igual	0,91	16/42
2	bayer_ticket_001351	OUTROS → SUPORTE_E_EQUIPAMENTOS	0,42	4/6
3	bayer_ticket_016056	SOLICITACAO_DE_SERVICO → igual	0,83	10/25

Fonte: Elaborado pelos autores.

esperado para a aquisição de *software*. O grafo mostra *licença* como o maior nó, ligado a *office* e *pacote*.

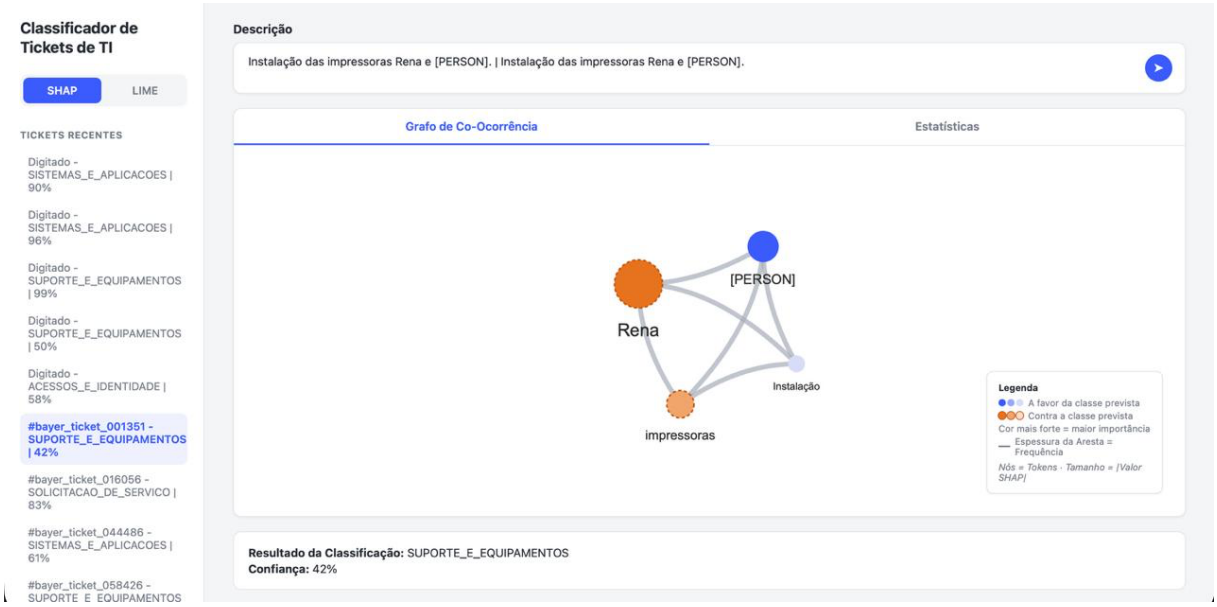
Figura 15 – Grafo de co-ocorrência do caso 1 (ache_ticket_001118), com pesos SHAP



Fonte: Elaborado pelos autores

No caso 2, o chamado tem uma única frase, sobre a instalação de duas impressoras, e a categoria real é OUTROS; o modelo previu SUPORTE_E_EQUIPAMENTOS com confiança de 0,42. O grafo tem apenas 4 nós. A palavra de maior peso, *Rena*, a marca de uma das impressoras, pesa contra a classe prevista (SHAP de −0,248; LIME de −0,290), assim como *impressoras* (−0,101 e −0,077), enquanto o marcador de anonimização *[PERSON]* pesa a favor (0,124 e 0,144). A explicação mostra que a decisão se apoiou em muito pouca evidência e que um marcador de anonimização influenciou a previsão, o que um analista pode questionar.

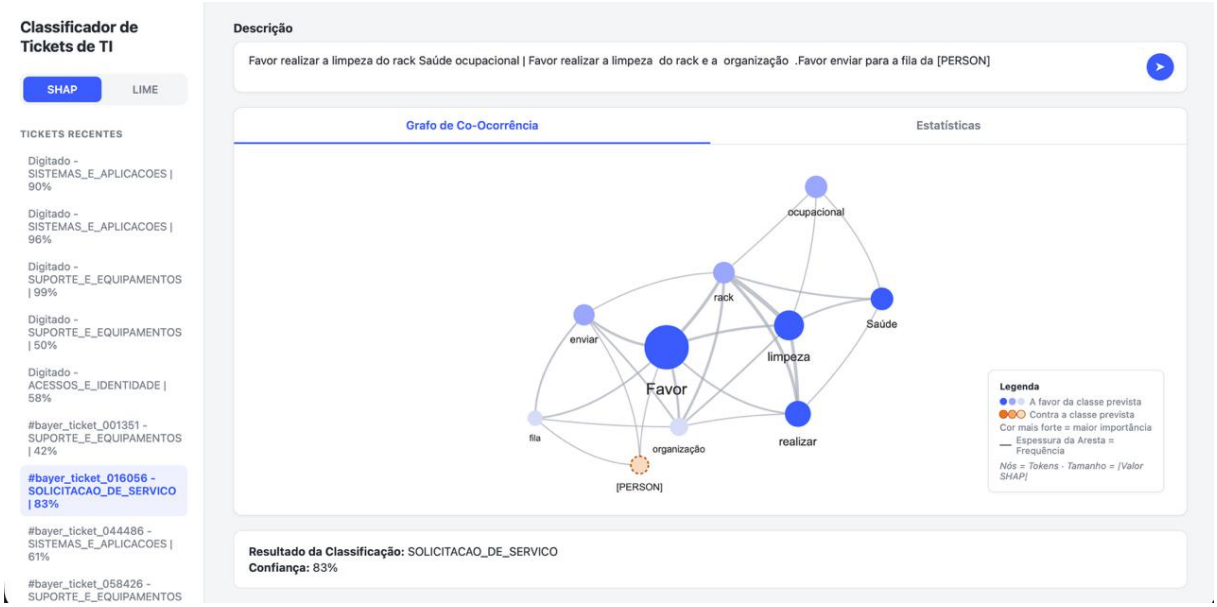
Figura 16 – Grafo de co-ocorrência do caso 2 (bayer_ticket_001351), com pesos SHAP



Fonte: Elaborado pelos autores

No caso 3, o chamado pede a limpeza de um *rack*. Os dois métodos concordam na palavra de maior peso, *Favor* (SHAP de 0,322; LIME de 0,346), típica de um pedido de serviço. Nas demais palavras, porém, divergem: *limpeza* tem peso de 0,165 no SHAP e de apenas 0,002 no LIME. O grafo, construído com os pesos do SHAP, e o do LIME, portanto, destacam conjuntos diferentes de palavras além da primeira, o que o analista pode verificar alternando o método na interface.

Figura 17 – Grafo de co-ocorrência do caso 3 (bayer_ticket_016056), com pesos SHAP



Fonte: Elaborado pelos autores

5.2 Análise crítica e comparação

Classificação. O BERTimbau superou o *baseline* SVM em 3,7 pontos percentuais de F1-macro (intervalo de 95% de 1,0 a 6,7), abaixo do critério de 5 pontos do TCC1. Os dados mostram que o ganho existe, mas não que ele atinja o critério: o intervalo inclui o 5, e a probabilidade de a diferença verdadeira ser de pelo menos 5 pontos é de cerca de 21%. O ganho vem da revocação e se concentra em poucas classes (SOFTWARE e SOLICITACAO_DE_SERVICO, além de duas classes pequenas), enquanto na maior classe os dois modelos empatam. Há ainda dois cuidados de leitura: o resultado na Aché é pouco informativo, pois seus chamados formam poucos grupos de texto, e na validação um único texto repetido 491 vezes inflaria a diferença para 11,9 pontos. Por isso, o resultado mais seguro é o do teste, com os grupos de texto reamostrados juntos.

Fidelidade. O SHAP e o LIME têm fidelidade bem acima do acaso, e a comparação chamado a chamado não distingue os métodos na *comprehensiveness* nem na *sufficiency*. Só no Tau-LOO há diferença, a favor do LIME (diferença média de $-0{,}059$ para o SHAP, com intervalo que exclui o zero). Isso difere do que se poderia esperar de um método com garantias teóricas como o SHAP, mas uma diferença em uma das três medidas não basta para afirmar que o LIME seja mais fiel.

Estabilidade. Salih et al. (2023) indicam que o SHAP tende a ser mais estável que o LIME. Neste trabalho, isso só vale quando o SHAP é calculado de forma exata, o que ocorre em textos curtos (até 10 palavras por campo), onde a explicação é idêntica em todas as repetições. Nos 9 chamados em que o SHAP foi estimado por amostragem, com orçamento de 500 consultas, o LIME foi mais estável (mesma palavra no topo em 93% das repetições, contra 68%). Uma explicação possível, que este trabalho não testou, é que 500 consultas são poucas para textos longos; o orçamento, portanto, é um parâmetro que troca tempo de processamento por estabilidade, e o resultado vale para o orçamento usado. A amostra de 12 chamados é pequena, e as conclusões são indicativas.

Concordância. Os dois métodos concordam bem acima do acaso (a mesma palavra no topo em 71% dos chamados), mas a concordância cai com o tamanho do texto. Assim, em chamados longos, a escolha do método muda de forma relevante quais palavras o analista vê como importantes.

Grafo. Nos três estudos de caso, a rastreabilidade se cumpriu: cada nó e cada aresta puderam ser levados ao chamado e aos pesos do SHAP e do LIME. Os casos também mostraram o uso prático da explicação: no caso 2, ela expôs que um erro do modelo se apoiou em muito pouca evidência e em um marcador de anonimização, e no caso 3, que os dois métodos concordam na palavra de maior peso, mas destacam palavras diferentes depois dela. A limitação é que três casos, escolhidos para ilustrar situações diferentes, mostram que o grafo é rastreável, mas não que ele melhore a decisão de um analista.

5.3 Discussão

A hipótese de pesquisa (Seção 1.3.3) tem duas partes, e os resultados as sustentam de forma diferente. A primeira afirma que o grafo ponderado pelos valores SHAP e LIME contribui para a auditabilidade ao permitir rastrear os termos que levaram à categoria. Essa parte foi sustentada nos termos em que foi definida, a rastreabilidade, nos três estudos de caso. Como a auditabilidade foi medida por rastreabilidade e não em um teste com analistas, o resultado mostra que o caminho da decisão pode ser reconstruído, e não que um analista o reconstrua com facilidade.

A segunda parte afirma que o SHAP e o LIME produzem explicações com fidelidade e estabilidade distintas. Na fidelidade, a diferença é pequena e só aparece em uma das três medidas, a favor do LIME. Na estabilidade, a diferença depende de como o SHAP é calculado: ele é perfeitamente estável quando exato e menos estável que o LIME quando estimado por amostragem com 500 consultas. A parte é, portanto, sustentada de forma condicional, e o resultado não sustenta a ideia de que um método seja melhor em todos os casos.

Em relação aos objetivos específicos, o ganho do BERTimbau sobre o *baseline* foi analisado (3,7 pontos, abaixo do critério de 5), e SHAP e LIME foram comparados em fidelidade, estabilidade e concordância. A consistência semântica das explicações foi avaliada apenas de forma qualitativa, nos casos: no caso 1, as palavras de maior peso (*licença* e *office*) são o vocabulário esperado para a classe, e no caso 2 a explicação mostra uma previsão apoiada em muito pouca evidência. Razma e Jurkus (2026) observam que o *fine-tuning* de um modelo como o BERT pode destacar termos coerentes com o significado do chamado; os casos analisados aqui mostram que isso ocorre em alguns chamados, mas não em todos.

Para a prática de suporte, os resultados sugerem que o grafo é mais útil como ferramenta de auditoria dos erros e das previsões de baixa confiança, como no caso 2, do que como justificativa de todos os acertos. Por fim, os resultados valem para os dados e a configuração usados (duas empresas, um treinamento, uma amostra de 98 chamados), e as limitações descritas no Capítulo 6 delimitam o alcance das conclusões.

6 Conclusão

6.1 Retomada do problema e dos objetivos

Este trabalho partiu de um problema: classificadores baseados em *transformers* acertam bem a categoria de um chamado de TI, mas não mostram por que a escolheram, o que dificulta a auditoria de suas decisões. A pergunta de pesquisa foi como se comparam o SHAP e o LIME quanto à fidelidade e à estabilidade das explicações de um classificador BERTimbau, e de que forma um grafo de co-ocorrência contribui para tornar o processo de classificação auditável.

Para respondê-la, os chamados de duas empresas (53.279 chamados, em 11 categorias) foram usados para treinar um *baseline* SVM com TF-IDF e ajustar o BERTimbau, e as decisões do BERTimbau foram explicadas por SHAP e LIME e representadas em um grafo. Os resultados, por objetivo específico, foram os seguintes.

1. **Ganho do BERTimbau sobre o *baseline*.** O BERTimbau obteve F1-macro de 0,628 contra 0,591 do SVM, uma diferença de 3,7 pontos percentuais (intervalo de confiança de 95% de 1,0 a 6,7), abaixo do critério de 5 pontos definido no TCC1. O ganho existe, mas os dados não mostram que atinja o critério, e ele se concentra em poucas classes.

2. **Comparação entre SHAP e LIME.** Em 98 chamados, os dois métodos tiveram fidelidade bem acima do acaso e não se distinguiram na *comprehensiveness* nem na *sufficiency*; só o Tau-LOO foi maior no LIME. Na estabilidade, o SHAP foi perfeito quando calculado de forma exata, mas menos estável que o LIME quando estimado por amostragem com 500 consultas. Os dois apontaram a mesma palavra no topo em 71% dos chamados, e a concordância caiu com o tamanho do texto.

3. **Auditabilidade pelo grafo.** Nos três estudos de caso, cada nó e cada aresta puderam ser levados ao chamado e aos pesos do SHAP e do LIME, o que atende à auditabilidade nos termos em que foi definida, a rastreabilidade. Não se mediu, porém, se analistas conseguem acompanhar a decisão a partir do grafo.

4. **Consistência semântica.** Foi avaliada apenas de forma qualitativa, nos casos: em um deles, as palavras de maior peso são o vocabulário esperado para a classe, e em outro a explicação expôs uma previsão apoiada em muito pouca evidência.

Assim, a hipótese de pesquisa foi sustentada na parte da rastreabilidade e, na parte da comparação entre os métodos, de forma condicional: nenhum dos dois foi melhor em todas as medidas.

6.2 Contribuições para a Computação

As contribuições deste trabalho para a área são as seguintes.

- **Comparação empírica entre SHAP e LIME em chamados de TI em português.** A literatura identificada não traz essa comparação para um BERT *fine-tuned* em chamados de suporte em português brasileiro. Os resultados mostram que a vantagem de estabilidade atribuída ao SHAP depende de como ele é calculado, e que a escolha do método importa mais em textos longos.
- **Protocolo de avaliação das explicações.** O trabalho reúne, em um mesmo procedimento, medidas de fidelidade, estabilidade e concordância, cada uma comparada ao valor esperado ao acaso, com intervalos de confiança por *bootstrap* pareado. O procedimento pode ser reaplicado a outros classificadores de texto.
- **Grafo de co-ocorrência com pesos SHAP e LIME como camada de auditabilidade.** O grafo liga cada decisão às palavras do chamado, à janela de co-ocorrência e aos pesos dos dois métodos, e foi implementado em uma interface *web* que serve de prova de conceito.
- **Evidência sobre o BERTimbau frente a um *baseline* clássico.** O trabalho mede o ganho do modelo neural em chamados reais de duas empresas, com a divisão dos dados agrupada pelo texto do chamado, de modo que textos repetidos não vazem entre treino e teste, e mostra como textos muito repetidos podem inflar a diferença entre os modelos.

6.3 Limitações e trabalhos futuros

Este trabalho tem limitações que delimitam o alcance de seus resultados.

Quanto aos dados, os chamados vêm de apenas duas empresas do setor farmacêutico, e a Bayer reúne 86% do conjunto de teste. Os chamados da Aché cobrem um único mês (abril de 2026) e têm muitos textos repetidos: os 716 chamados de teste dessa empresa formam apenas 337 grupos de texto, de modo que o resultado na Aché traz menos informação do que o número de chamados sugere. A classe OUTROS tem apenas 12 chamados no teste e nenhum foi acertado, então o desempenho nela não pôde ser avaliado de forma conclusiva. As categorias das duas empresas foram harmonizadas em uma taxonomia única, e os resultados dependem dessa escolha.

Além disso, dos 189.909 chamados da Bayer que restaram após a limpeza, 143.325 não estavam em português e foram descartados pelo filtro de idioma, e não traduzidos; traduzi-los poderia ampliar a base, ao custo de introduzir ruído de tradução no vocabulário técnico.

Quanto à classificação, os resultados vêm de um único treinamento do BERTimbau (semente 42), sem repetição com outras sementes. O intervalo de confiança da diferença para o *baseline* SVM (de 1,0 a 6,7 pontos) inclui tanto ganhos pequenos quanto o critério de 5 pontos, e cerca de 10,2% dos pares de texto ultrapassam o limite de 256 *tokens*, de modo que o trecho excedente não chega ao modelo.

Quanto à escolha do modelo, o trabalho usou o BERTimbau, um modelo de 2020, e não avaliou modelos generalistas mais recentes (LLMs), nem abordagens com recuperação de contexto (RAG, *Retrieval-Augmented Generation* – Geração Aumentada por Recuperação) ou explicações em texto geradas pelo próprio modelo, como as de *Chain-of-Thought* (Cadeia de Raciocínio). O BERTimbau foi mantido por ser específico para o português e por permitir calcular, em máquinas locais, as centenas de consultas ao modelo que o SHAP e o LIME exigem por explicação; com um LLM, esse custo seria bem maior. Já as explicações em texto de um LLM teriam de ter sua fidelidade ao processo de decisão avaliada com medidas como as usadas aqui.

Quanto às explicações, o SHAP e o LIME foram avaliados em uma amostra de 98 chamados, e a estabilidade em apenas 12 deles, explicados 5 vezes cada e escolhidos entre os que têm ao menos oito palavras distintas, o que impede estender esse resultado a chamados curtos. Em 79 dos 166 campos explicados, o SHAP foi estimado por amostragem, com orçamento de 500 consultas, o mesmo número de amostras do LIME; orçamentos maiores podem alterar a estabilidade medida. Em alguns chamados, o modelo linear local do LIME ajusta pouco o comportamento do classificador (o R^2 local teve mediana de 0,68 e mínimo de 0,11). Além disso, as medidas de fidelidade foram implementadas pelos autores, seguindo a definição de Attanasio et al. (2023), e por isso seus valores podem não ser diretamente comparáveis aos de estudos que usam a biblioteca *ferret*. A fidelidade e a estabilidade não foram calculadas para o SVM, que é interpretável por construção, e a comparação das explicações entre os cenários A e B fica como trabalho futuro.

Quanto ao grafo, a auditabilidade foi avaliada pela rastreabilidade, em estudos de caso, e não em um teste com analistas de TI. O trabalho, portanto, mostra que cada nó e cada aresta podem ser levados de volta ao chamado, mas não que analistas conseguem, na prática, acompanhar a decisão do classificador a partir do grafo.

Como trabalhos futuros, sugerem-se: (i) repetir o treinamento com outras sementes e incluir chamados de outras empresas e de períodos mais longos; (ii) explicar uma amostra maior de chamados, com mais amostras e consultas por explicação, para medir a estabilidade com mais precisão; (iii) avaliar o grafo em um teste com analistas de TI; (iv) comparar o grafo ponderado por SHAP e por LIME com outras formas de apresentar as explicações; e (v) comparar o BERTimbau com modelos generalistas mais recentes e avaliar abordagens com RAG e explicações por cadeia de raciocínio.

Referências

ATTANASIO, G. et al. ferret: a framework for benchmarking explainers on transformers. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. [S.l.]: Association for Computational Linguistics, 2023. 48, 73

AXELOS. *ITIL Foundation, ITIL 4 Edition*. 1. ed. Norwich: TSO, 2019. ISBN 9780113316076. 20

BORGES, B. R. *Análise comparativa de algoritmos de classificação de texto*. Dissertação (Trabalho de Conclusão de Curso – Sistemas de Informação) — Universidade Federal de Uberlândia, 2024. Disponível em: <https://repositorio.ufu.br/handle/123456789/43701>. 29

BRASIL. *Lei n.º 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 26 ago. 2026. 59

DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. [s.n.], 2019. Disponível em: <https://arxiv.org/abs/1810.04805>. 12, 18, 24, 25, 26, 27, 28

HARUN, N. A.; HUSPI, S. H.; IAHAD, N. A. Question classification framework for helpdesk ticketing support system using machine learning. *International Journal on Informatics and Communication Technology (IJIC)*, v. 13, n. 1, 2023. Disponível em: <https://ijic.utm.my/index.php/ijic/article/view/388>. 12, 21

JACOVI, A.; GOLDBERG, Y. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020. p. 4198–4205. Disponível em: <https://aclanthology.org/2020.acl-main.386/>. 19

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <https://arxiv.org/abs/1705.07874>. 13, 18, 19, 33, 34, 36

MIKOLOV, T. et al. Efficient estimation of word representations in vector space. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2013. Disponível em: <https://arxiv.org/abs/1301.3781>. 23

MUSTAFA, S.; Hama Saeed, M. Empowering text classification with NLP and explainable AI for enhanced interpretability. *Journal of Electrical Systems and Information Technology*, Springer, v. 12, n. 81, 2025. Disponível em: <https://link.springer.com/article/10.1186/s43067-025-00273-2>. 13, 15, 32, 35

PARAMESH, S. P.; SHREEDHARA, K. S. A deep learning based IT service desk ticket classifier using CNN. *ICTACT Journal on Soft Computing*, v. 13, n. 1, p. 2805–2812, out. 2022. ISSN 2229-6956. 22

PEREIRA, L. S. B. et al. Machine learning for classification of IT support tickets. In: *International Conference on Cyber Management and Engineering (CyMaEn)*. IEEE, 2023. Disponível em: <https://www.researchgate.net/publication/368906495>. 12, 13, 21, 22, 43

RAZMA, A.; JURKUS, R. AI-based classification of IT support requests in enterprise service management systems. *Systems*, MDPI, v. 14, n. 2, 2026. Disponível em: <https://doi.org/10.3390/systems14020223>. 12, 13, 19, 23, 30, 36, 40, 41, 43, 70

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. "why should I trust you?": Explaining the predictions of any classifier. In: *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016. Disponível em: <https://arxiv.org/abs/1602.04938>. 13, 18, 19, 30, 31, 32, 37

SALIH, A. et al. A perspective on explainable artificial intelligence methods: SHAP and LIME. *Advanced Intelligent Systems*, Wiley, 2023. Disponível em: <https://arxiv.org/abs/2305.02012>. 13, 18, 19, 30, 31, 33, 35, 69

SOUZA, F.; NOGUEIRA, R.; LOTUFO, R. BERTimbau: Pretrained BERT models for Brazilian Portuguese. In: *Brazilian Conference on Intelligent Systems (BRACIS)*. Springer, 2020. Disponível em: <https://www.researchgate.net/publication/345395208>. 13, 18, 28, 29, 46

TALEBI, S.; ABDOLVAND, N. Building safer social spaces: Body shaming detection using RoBERTa, LIME and term co-occurrence graph. *International Journal of Web Research (IJWR)*, 2025. Disponível em: <https://doaj.org/article/3a3f1e1beed847909118aac97aacce05>. 13, 39, 40

VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <https://arxiv.org/abs/1706.03762>. 18, 24, 25

WAHBA, Y. M. *A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets)*. Tese (Tese de Doutorado em Ciência da Computação) — University of Western Ontario, 2023. Disponível em: <https://ir.lib.uwo.ca/etd/9192>. 12, 15, 20, 23, 40, 42, 43, 46

YANG, X.; CUI, Y. BERT-enhanced text graph neural network for classification. *Entropy*, MDPI, v. 23, n. 11, 2021. Disponível em: <https://www.mdpi.com/1099-4300/23/11/1536>. 38, 39

YAO, L.; MAO, C.; LUO, Y. Graph convolutional networks for text classification. In: *AAAI Conference on Artificial Intelligence*. [s.n.], 2019. Disponível em: <https://arxiv.org/abs/1809.05679>. 13, 18, 37, 38

YOKOTA, B. *Aprendizado de máquina baseado em Classificação hierárquica de textos aplicado ao contexto chamados de suporte*. Dissertação (MBA em Inteligência Artificial e Big Data) — Universidade de São Paulo – USP/ICMC, 2025. Disponível em: <https://bdta.abcd.usp.br/directbitstream/43ebe5d7-98d6-4357-915e-71486db593d5/Beatriz_Yokota.pdf>. 13, 22, 41, 43

ZEMP, M. *Text Classification of Service Desk Tickets*. Dissertação (Master Thesis – MAS Data Science) — Zurich University of Applied Sciences (ZHAW), 2021. Disponível em: <https://www.zhaw.ch/storage/shared/upload/MAS21_Ticket_Classification_Zemp.pdf>. 12, 17, 19, 20, 29, 30, 40, 42, 43

Apêndices

APÊNDICE A – Cronograma de atividades

O cronograma a seguir apresenta as fases do TCC2, com as datas em que foram concluídas e o status atual.

Tabela 14 – Cronograma do TCC2, com datas e status

Fase	Período	Status
1. Dados e modelo	setembro a 04/10/2026	Concluída (modelo final treinado em 04/10)
2. XAI e grafo	outubro a 01/11/2026	Concluída (rodada oficial do SHAP e do LIME em 01/11)
3. Sistema e interface	até 05/11/2026	Concluída (prova de conceito funcionando em 05/11)
4. Escrita do documento	02/11 a 07/11/2026	Concluída (capítulos 1 a 6 redigidos)
5. Prévia e entrega	10/11 e 24/11/2026	Prévia da banca e entrega final agendadas

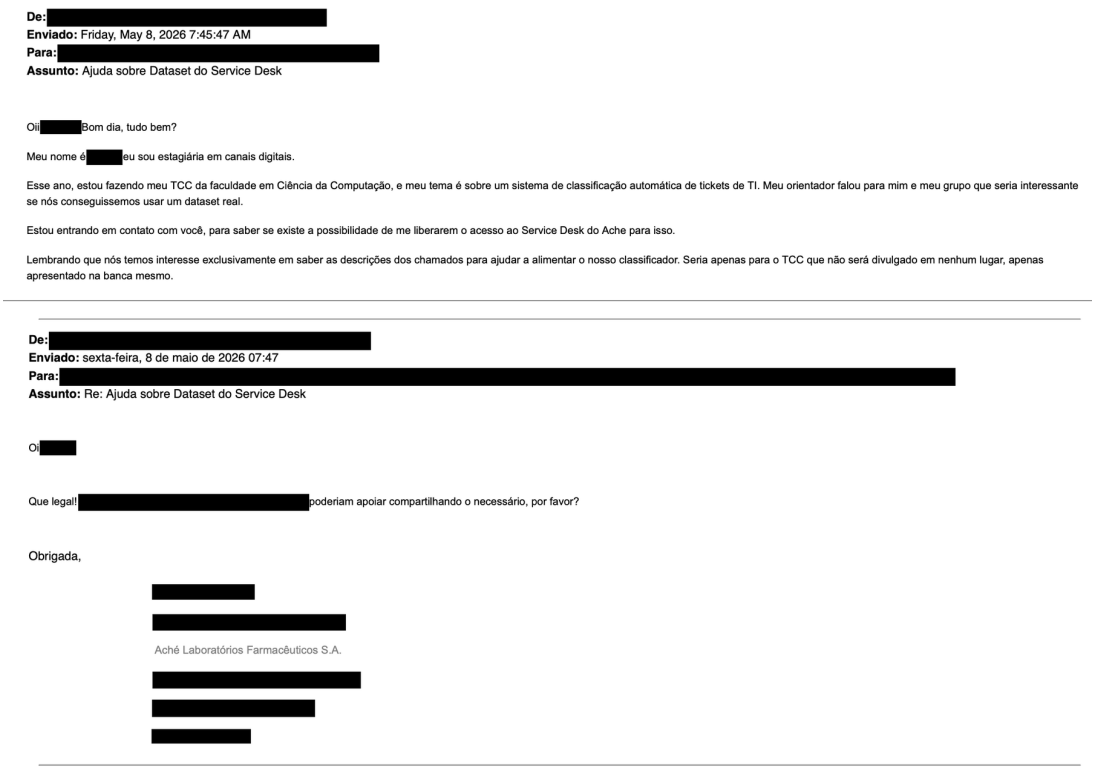
Fonte: Elaborado pelos autores. Em relação ao plano inicial, a interface passou a ser servida com *Flask*, em vez de *FastAPI*.

Anexos

ANEXO A – Autorização de uso de dados — Aché Laboratórios Farmacêuticos

Troca de e-mails em que a Aché Laboratórios Farmacêuticos, por meio de sua equipe de suporte de TI e do prestador do serviço, concorda em compartilhar a base de chamados de suporte técnico para fins acadêmicos neste trabalho. Nomes, endereços de e-mail, telefones e assinaturas dos participantes foram ocultados, mantendo-se a data, o assunto e o conteúdo da conversa. As mensagens aparecem em ordem cronológica: o pedido dos autores e a concordância da empresa (Figura 18) e o envio da base de chamados (Figura 19).

Figura 18 – Autorização de uso de dados concedida pela Aché Laboratórios Farmacêuticos (1 de 2): pedido e concordância



De: [redacted]
Enviado: Friday, May 8, 2026 7:45:47 AM
Para: [redacted]
Assunto: Ajuda sobre Dataset do Service Desk

Oii [redacted] Bom dia, tudo bem?

Meu nome é [redacted] eu sou estagiária em canais digitais.

Esse ano, estou fazendo meu TCC da faculdade em Ciência da Computação, e meu tema é sobre um sistema de classificação automática de tickets de TI. Meu orientador falou para mim e meu grupo que seria interessante se nós conseguissemos usar um dataset real.

Estou entrando em contato com você, para saber se existe a possibilidade de me liberarem o acesso ao Service Desk do Ache para isso.

Lembrando que nós temos interesse exclusivamente em saber as descrições dos chamados para ajudar a alimentar o nosso classificador. Seria apenas para o TCC que não será divulgado em nenhum lugar, apenas apresentado na banca mesmo.

De: [redacted]
Enviado: sexta-feira, 8 de maio de 2026 07:47
Para: [redacted]
Assunto: Re: Ajuda sobre Dataset do Service Desk

Oi [redacted]

Que legal! [redacted] poderiam apoiar compartilhando o necessário, por favor?

Obrigada,

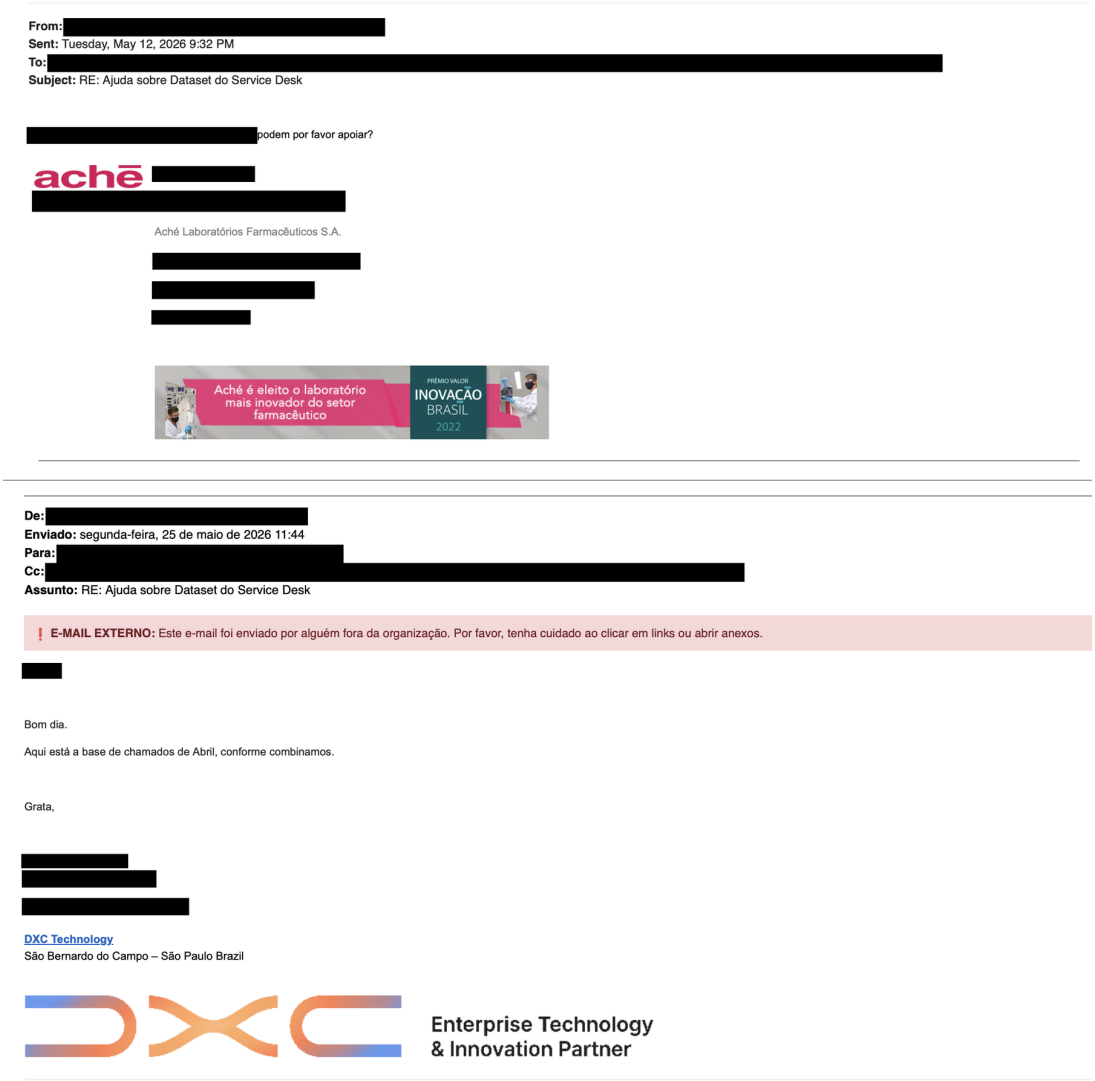
[redacted]

Aché Laboratórios Farmacêuticos S.A.

[redacted]

Fonte: Correspondência recebida pelos autores (2026)

Figura 19 – Autorização de uso de dados concedida pela Aché Laboratórios Farmacêuticos (2 de 2): acompanhamento e envio da base



Fonte: Correspondência recebida pelos autores (2026)

ANEXO B – Autorização de uso de dados — Bayer S.A.

Mensagem de e-mail em que a Bayer S.A. autoriza o uso de chamados de suporte técnico para fins acadêmicos neste trabalho, mediante anonimização dos registros. Nomes, e-mails, ramais e assinaturas dos remetentes foram removidos da imagem, mantendo-se apenas o conteúdo da autorização e a identificação da empresa.

Figura 20 – Autorização de uso de dados concedida pela Bayer S.A.



De: [redacted]
Enviado: Friday, May 8, 2026 7:45:47 AM
Para: [redacted]
Assunto: Ajuda sobre Dataset do Service Desk

Oi [redacted] Bom dia, tudo bem?

Meu nome é [redacted] eu sou estagiária em canais digitais.

Esse ano, estou fazendo meu TCC da faculdade em Ciência da Computação, e meu tema é sobre um sistema de classificação automática de tickets de TI. Meu orientador falou para mim e meu grupo que seria interessante se nós conseguissemos usar um dataset real.

Estou entrando em contato com você, para saber se existe a possibilidade de me liberarem o acesso ao Service Desk do Ache para isso.

Lembrando que nós temos interesse exclusivamente em saber as descrições dos chamados para ajudar a alimentar o nosso classificador. Seria apenas para o TCC que não será divulgado em nenhum lugar, apenas apresentado na banca mesmo.

De: [redacted]
Enviado: sexta-feira, 8 de maio de 2026 07:47
Para: [redacted]
Assunto: Re: Ajuda sobre Dataset do Service Desk

Oi [redacted]

Que legal! [redacted] poderiam apoiar compartilhando o necessário, por favor?

Obrigada,

[redacted]

Aché Laboratórios Farmacêuticos S.A.

[redacted]

Fonte: Correspondência recebida pelos autores (2026)